

JL701N_soundbox_SDK_介绍 用户手册

珠海市杰理科技股份有限公司

Zhuhai Jieli Technology Co.,LTD

版权所有，未经许可，禁止外传

目录

Chapter 1	SDK 功能说明	5
1.1	编写目的	5
1.2	模式介绍	5
1.3	细分功能介绍	5
1.4	是否使用配置文件的参数配置	9
1.5	功能选择注意事项	10
Chapter 2	VM 接口的使用	11
2.1	编写目的	11
2.2	详细说明	11
Chapter 3	APP 模式管理	14
3.1	编写目的	14
3.2	模式任务	14
3.3	模式流程的详细介绍（以上电进入 powerno 模式为例）	16
3.4	APP 模式管理函数接口介绍	18
3.5	增加 APP 模式例子（以 music 为例）	22
Chapter 4	APP 消息管理	25
4.1	系统事件管理	25
4.2	提供的 APP 消息处理函数说明	26
Chapter 5	SD、USB 复用 IO 介绍	30
5.1	编写目的	30
5.2	sd 复用 api 说明	30
5.3	sd cmd 复用 linein ad 检测	31
5.4	sd clk 复用 iokey 检测	32
5.5	sd data 复用 usb dm	32
Chapter 6	线程优先级和时钟	34
6.1	线程优先级设置	34
6.2	时钟设置	35
Chapter 7	Audio 模块使用	37
7.1	编写目的	37
7.2	在线 EQ 调试	37
7.3	DAC 配置	51
7.3.1	DAC 输出通道配置	51
7.3.2	DAC 高低压模式选择	53

7.3.3 DAC 输出幅度	53
7.3.4 DAC 使用范例	56
7.4 ADC 配置	56
7.4.1 ADC 使用范例	56
7.4.2 通话过程 mic 数据获取	57
7.4.3 省电容 mic 使用	57
7.5 音量配置	58
7.5.1 音量调节方式	58
7.5.2 K 歌音箱音量配置指引	59
7.6 编解码配置	59
7.6.1 解码声道配置	59
7.6.2 编解码格式配置	62
7.7 音频同步配置	62
7.8 通话算法开发配置	63
7.8.1 算法离线开发	63
7.8.2 第三方算法开发	64
7.8.3 通话参数调试	64
7.8.4 双 mic 降噪 (ENC)	65
7.8.5 神经网络降噪 DNS	65
7.8.6 双 MIC 话务耳机	66
7.9 语音识别配置	67
7.10 音效配置	69
7.10.1 音效支持列表	69
7.10.2 在线调试配置	70
7.10.3 音效配置	70
7.10.4 音效模块使用说明	72
Chapter 8 客户常见问题反馈解答	73
Q1:	73
Q2:	73

修改记录

版本	更新日期	描述
1.0	2022 / 3 / 10	JL701N 用户手册
更新:	● 建立初始版本 ● 定义文档格式	



	通话	回拨
		接听拒接
		声卡切换
		三方通话
	spp	
	hid	hid 触摸
	ble	弹窗、透传
		可支持开发 app
	tw s 对箱	暂不支持
		暂不支持
		暂不支持
		暂不支持
	emitter 发射器	暂不支持
		暂不支持
		暂不支持
		暂不支持
	后台	fm、蓝牙共存
解码	格式	CVSD\MSBC\SBC\AAC\G729\MP3\WMA\FLAC\APE\M4A\AMR\DTS\G726\MIDI
	加密播放	
音频播放	按路径播放	
	音频位置信息	文件夹总数、文件数、当前文件在当前文件夹序
	播放模式	单曲、循环、随机……
	文件系统	fat12、fat16、fat32、extfat
编码	格式	CVSD、MSBC\G726\MP3\ADPCM\SBC
音源输入	iis	暂不支持
	hdmi	暂不支持
	spdif	暂不支持

	linein	数字
	pc	
音效	eq、drc\在线调试	支持 20 段
	音效、在线调试	魔音、啸叫抑制、变速、闪避、消人声 变调、高低音、电音……
fm	外置 fm 收音	
rtc	时钟、闹钟	
外设	usb	pc (massstorage、speaker、mic、hid)
	Sd	文件系统、pc 出盘符
提示音	格式	mp3\wtg\msbc\sbc\sin\nty
按键	iokey	
	adkey	
	irkey	红外
	touchkey	触摸
	rdeckey	旋码器
显示	led	
升级	U 盘	支持 uboot ui、io 保持、ufw 文件只识别后缀
	sd 卡	
	串口	
	测试盒	
	烧写器	
其他	内置充电	
	系统时钟	最高 160M

	不可屏蔽中断优 优先级	暂不支持



1.4 是否使用配置文件的参数配置

默认 sdk 是从配置文件读取配置来设置灯状态和提示音，如果不想从配置文件读取配置，只需修改宏 USE_CONFIG_STATUS_SETTING 为 0。

```
25 #define USE_CONFIG_STATUS_SETTING      1      //状态设置，包括灯状态和提示音
26 #define USE_CONFIG_AUDIO_SETTING      0      //音频设置
27 #define USE_CONFIG_MIC_TYPE_SETTING   0      //MIC类型设置
28 #define USE_CONFIG_AUTO_OFF_SETTING   0      //自动关机时间设置
29 #define USE_CONFIG_COMBINE_VOL_SETTING 0      //联合音量读配置
30
```

然后默认配置在每个 board.c 里面设置：

```
41 /*各个状态下默认的闪光灯方式和提示音设置，如果USER_CFG中设置了USE_CONFIG_STATUS_SETTING为1，则会从配置文件读取对应的配置来填充结构体*/
42 STATUS_CONFIG status_config = {
43     //灯状态设置
44     .led = {
45         .charge_start = PWM_LED1_ON,
46         .charge_full = PWM_LED0_ON,
47         .power_on = PWM_LED0_ON,
48         .power_off = PWM_LED1_FLASH_THREE,
49         .lowpower = PWM_LED1_SLOW_FLASH,
50         .max_vol = PWM_LED_NULL,
51         .phone_in = PWM_LED_NULL,
52         .phone_out = PWM_LED_NULL,
53         .phone_activ = PWM_LED_NULL,
54         .bt_init_ok = PWM_LED0_LED1_SLOW_FLASH,
55         .bt_connect_ok = PWM_LED0_ONE_FLASH_5S,
56         .bt_disconnect = PWM_LED0_LED1_FAST_FLASH,
57         .tw_s_connect_ok = PWM_LED0_LED1_FAST_FLASH,
58         .tw_s_disconnect = PWM_LED0_LED1_SLOW_FLASH,
59     },

```

注意：如果想生成的 fw 文件能单独配置某些选项，要把对应的宏打开。

1.5 功能选择注意事项

JL701N_SOUNDBOX_SDK 功能比较全面独立，开发人员可以按照需求组合功能，但是仍有一些功能，因为资源和速度的限制，SDK 做了一些编译限制，例如：

```
20 //*****  
21 // 编译警告  
22 //*****  
23 #if ((TCFG_COMM_TYPE == TCFG_UART_COMM) && TCFG_ONLINE_ENABLE) && (TCFG_PC_ENABLE || TCFG_UDISK_ENABLE)  
24 #error "eq online adjust enable, please close usb marco"  
25 #endif// ((TCFG_ONLINE_ENABLE) && (TCFG_PC_ENABLE || TCFG_UDISK_ENABLE))  
26  
27 #if ((TCFG_COMM_TYPE == TCFG_UART_COMM) && TCFG_SOUNDBOX_TOOL_ENABLE) && TCFG_PC_ENABLE  
28 #error "soundbox tool enable, please close pc marco"  
29 #endif// (TCFG_SOUNDBOX_TOOL_ENABLE) && (TCFG_PC_ENABLE)  
30  
31 #if TCFG_UI_ENABLE  
32 #if ((TCFG_SPI_LCD_ENABLE && TCFG_CODE_FLASH_ENABLE) && (TCFG_FLASH_DEV_SPI_HW_NUM == TCFG_TFT_LCD_DEV_SPI_HW_NUM))  
33 #error "flash spi port == lcd spi port, please close one !!!"  
34 #endif//((TCFG_SPI_LCD_ENABLE && TCFG_CODE_FLASH_ENABLE) && (TCFG_FLASH_DEV_SPI_HW_NUM == TCFG_TFT_LCD_DEV_SPI_HW_NUM))  
35 #endif//TCFG_UI_ENABLE  
36  
37 #if((TRANS_DATA_EN + DUEOS_DMA_EN + ANCS_CLIENT_EN) > 1)  
38 #error "they can not enable at the same time,just select one!!!"  
39 #endif//((TRANS_DATA_EN && DUEOS_DMA_EN)  
40  
41 #if (TCFG_DEC2TWS_ENABLE && (TCFG_MIC_EFFECT_ENABLE ||TCFG_APP_RECORD_EN || TCFG_APP_RTC_EN ||TCFG_DRC_ENABLE))  
42 #error "对箱支持音源转发，请关闭混响录音等功能 !!!"  
43 #endif// (TCFG_DEC2TWS_ENABLE && (TCFG_MIC_EFFECT_ENABLE ||TCFG_APP_RECORD_EN || TCFG_APP_RTC_EN ||TCFG_DRC_ENABLE))  
44  
45 #if (TCFG_MIC_EFFECT_ENABLE && (TCFG_DEC_APE_ENABLE || TCFG_DEC_FLAC_ENABLE || TCFG_DEC_DTS_ENABLE))  
46 // #error "无损格式+混响暂时不支持同时打开 !!!"  
47 #endif//((TCFG_MIC_EFFECT_ENABLE && (TCFG_DEC_APE_ENABLE || TCFG_DEC_FLAC_ENABLE || TCFG_DEC_DTS_ENABLE))  
48  
49 #if (TCFG_REVERB_DODGE_EN && (USER_DIGITAL_VOLUME_ADJUST_ENABLE == 0))  
50 #error "使用闪避功能，打开自定义数字音量调节 !!!"  
51 #endif  
52  
53 #if ((TCFG_NORFLASH_DEV_ENABLE || TCFG_NOR_FS_ENABLE) && TCFG_UI_ENABLE)  
54 #error "引脚复用问题，使用norflash需要关闭UI !!!"  
55 #endif  
56  
57 #if ((TCFG_APP_RECORD_EN) && (TCFG_USER_TWS_ENABLE))  
58 #error "TWS 暂不支持录音功能"  
59 #endif  
60  
61 #if ((AUDIO_EQUALLOUDNESS_CONFIG) && (SYS_VOL_TYPE == VOL_TYPE_ANALOG))  
NORMAL master apps/soundbox/board/br28/board_config.h
```

Chapter 2 VM 接口的使用

2.1 编写目的

该文档主要描述 JL701N_SDK VM 使用方法及开发中注意的一些问题，为用户进行二次开发提供参考。

2.2 详细说明

Vm 读写接口：

读接口：

```
int syscfg_read(u16 item_id, void *buf, u16 len);
```

写接口：

```
int syscfg_write(u16 item_id, void *buf, u16 len);
```

参数说明：

item_id: vm id 范围 1 ~ 60

*buf: 读写指针

Len: 长度

比如保存 USB MIC GAIN 和读 USB MIC GAIN 的 vm 操作：

1. 确定 vm id 号

```

4 //=====//
5 //                      与APP CASE相关配置项[1 ~ 60]                      //
6 //=====//
7 #define      VM_FM_EMITTER_FREQ          1
8 // #define    VM_FM_EMITTER_DIG_VOL      2
9 // #define    VM_MUSIC_LAST_DEV          3
10 // #define    VM_WAKEUP_PND              4
11 #define      VM_USB_MIC_GAIN              5
12 #define      VM_ALARM_0                  6
13 #define      VM_ALARM_1                  7
14 #define      VM_ALARM_2                  8
15 #define      VM_ALARM_3                  9
16 #define      VM_ALARM_4                 10
17 #define      VM_ALARM_MASK              11
18 #define      VM_ALARM_NAME_0            12
19 #define      VM_ALARM_NAME_1            13
20 #define      VM_ALARM_NAME_2            14
21 #define      VM_ALARM_NAME_3            15
22 #define      VM_ALARM_NAME_4            16
23 #define      VM_FM_INFO                  17
24 // #define    VM_RTC_TRIM                  18
25
26 #define      CFG_RCSP_ADV_HIGH_LOW_VOL   19
27 #define      CFG_RCSP_ADV_EQ_MODE_SETTING 20
28 #define      CFG_RCSP_ADV_EQ_DATA_SETTING 21
29 #define      ADV_SEQ_RAND                 22
30 #define      CFG_RCSP_ADV_TIME_STAMP      23
31 #define      CFG_RCSP_ADV_WORK_SETTING    24
32 #define      CFG_RCSP_ADV_MIC_SETTING     25
33 #define      CFG_RCSP_ADV_LED_SETTING     26
34 #define      CFG_RCSP_ADV_KEY_SETTING     27
35 #define      CFG_RCSP_MISC_DRC_SETTING    28
36 #define      CFG_RCSP_MISC_REVERB_ON_OFF 29
37 #define      VM_ALARM_RING_NAME_0        30
38 #define      VM_ALARM_RING_NAME_1        31
39 #define      VM_ALARM_RING_NAME_2        32
NORMAL ▶ master ▶ ~/work/master/SDK/apps/soundbox/include/user_cfg_id.h

```

2.写 USB MIC GAIN

```

287     if (++usb_mic_gain_save_cnt >= 5) {
288         sys_hi_timer_del(usb_mic_gain_save_timer);
289         usb_mic_gain_save_timer = 0;
290         usb_mic_gain_save_cnt = 0;
291         local_irq_enable();
292         printf("USB_GAIN_SAVE\n");
293         syscfg_write(VM_USB_MIC_GAIN, &app_var.usb_mic_gain, 1);
294         return;

```

3.读 USB MIC GAIN

```
319      //// pc usb mic gain
320      s8 usb_mic_gain = -1;
321      ret = syscfg_read(VM_USB_MIC_GAIN, &usb_mic_gain, 1);
322      if (ret < 0) {
323          usb_mic_gain = 7;
324      }
325      app_var.usb_mic_gain = usb_mic_gain;
326      log_info("usb_mic_gain: %d\n", app_var.usb_mic_gain);
327
```


Chapter 3 APP 模式管理

3.1 编写目的

该文档主要描述 JL701N_SDK APP 模式使用方法及开发中注意的一些问题，为用户进行二次开发提供参考。

3.2 模式任务

1. 模式任务 app_core 创建，所有的 APP 模式在此任务里执行处理

```
int main()
{
    wdt_close();

    os_init();

    setup_arch();

    board_early_init();

    task_create(app_task_handler, NULL, "app_core");

    os_start();

    local_irq_enable();

    while (1) {
        asm("idle");
    }

    return 0;
}
```

RMAL master apps/soundbox/common/init.c

2. APP 模式处理函数 app_task_loop()

```
void app_task_loop()
{
    while (1) {
        switch (app_curr_task) {
            case APP_POWERON_TASK:
                log_info("APP_POWERON_TASK \n");
                app_poweron_task();
                break;
            case APP_POWEROFF_TASK:
                log_info("APP_POWEROFF_TASK \n");
                app_poweroff_task();
                break;
            case APP_BT_TASK:
                log_info("APP_BT_TASK \n");
                app_bt_task();
                break;
            case APP_MUSIC_TASK:
                log_info("APP_MUSIC_TASK \n");
                app_music_task();
                break;
            case APP_FM_TASK:
                log_info("APP_FM_TASK \n");
                app_fm_task();
                break;
            case APP_RECORD_TASK:
                log_info("APP_RECORD_TASK \n");
                app_record_task();
                break;
            case APP_LINEIN_TASK:
                log_info("APP_LINEIN_TASK \n");
                app_linein_task();
                break;
            case APP_RTC_TASK:
                log_info("APP_RTC_TASK \n");
                app_rtc_task();
                break;
            case APP_PC_TASK:
                log_info("APP_PC_TASK \n");
                app_pc_task();
                break;
            case APP_SPDIF_TASK:
                log_info("APP_SPDIF_TASK \n");
                app_spdif_task();
                break;
            case APP_IDLE_TASK:
                log_info("APP_IDLE_TASK \n");
                app_idle_task();
                break;
        }
    }
}
```

RMAL master apps/soundbox/app_main.c

3. APP 模式值

```
enum {  
    APP_POWERON_TASK    = 1,  
    APP_POWEROFF_TASK   = 2,  
    APP_BT_TASK          = 3,  
    APP_MUSIC_TASK       = 4,  
    APP_FM_TASK          = 5,  
    APP_RECORD_TASK      = 6,  
    APP_LINEIN_TASK      = 7,  
    APP_RTC_TASK         = 8,  
    APP_PC_TASK          = 9,  
    APP_SPDIF_TASK       = 10,  
    APP_IDLE_TASK        = 11,  
    APP_TASK_MAX_INDEX,  
};
```

3.3 模式流程的详细介绍（以上电进入 poweron 模式为例）

1.APP 模式设置

上电设置 app 模式值

```
/* endless_loop_debug_int(); */  
ui_update_status(STATUS_POWERON);  
app_curr_task = APP_POWERON_TASK;  
}  
  
app_task_loop();
```

AL master apps/soundbox/app_main.c

2.Poweron 模式进入


```
void app_task_loop()
{
    while (1) {
        switch (app_curr_task) {
            case APP_POWERON_TASK:
                log_info("APP_POWERON_TASK \n");
                app_poweron_task();
                break;
            case APP_POWEROFF_TASK:
                log_info("APP_POWEROFF_TASK \n");
                app_poweroff_task();
                break;
            case APP_BT_TASK:
                log_info("APP_BT_TASK \n");
                app_bt_task();
                break;
            case APP_MUSIC_TASK:
                log_info("APP_MUSIC_TASK \n");
```

3.APP 模式切换

(1) 设置下一模式值

```
static int power_on_init(void)
{
    ///有些需要在开机提示完成之后再初始化的东西， 可以在这里初始化
    #if (TCFG_SPI_LCD_ENABLE)
        lcd_ui_power_on();//由ui决定切换的模式
        return 0;
    #endif

    printf("----->%s, %d\n", FUNCTION, __LINE__);
    #if TCFG_APP_BT_EN
        app_task_switch_to(APP_BT_TASK);
    #else
        app_task_switch_to(APP_MUSIC_TASK);
    #endif

    return 0;
}
```

(2) 推出 APP_MSG_SWITCH_TASK 切换消息

```

0 int app_task_switch_to(u8 app_task)
1 {
2     if (app_curr_task == app_task) {
3         return false;
4     }
5
6     if (!app_task_switch_check(app_task)) {
7         return false;
8     }
9
10    if (!app_task_switch_exit_check(app_curr_task)) {
11        return false;
12    }
13
14    printf("cur --- %x \n", app_curr_task);
15    printf("new +++ %x \n", app_task);
16
17    /* if(app_next_task) */
18    /* printf("app_task_switch_to busy \n"); */
19    #if (defined SMART_BOX_EN) && (SMART_BOX_EN)
20    extern void function_change_inform(u8 app_mode, u8 ret);
21    function_change_inform(app_task, TRUE);
22    #endif
23
24    app_prev_task = app_curr_task;
25    app_next_task = app_task;
26    app_task_put_usr_msg(APP_MSG_SWITCH_TASK, 0);
27
28    return TRUE;
29 }

```

4.poweron 模式退出，跳出 while（1）循环

```

while (1) {
    app_task_get_msg(msg, ARRAY_SIZE(msg), 1);
    switch (msg[0]) {
        case APP_MSG_SYS_EVENT:
            if (poweron_sys_event_handler((struct sys_event *) (msg + 1)) == false) {
                app_default_event_deal((struct sys_event *) (&msg[1])); //由common统一处理
            }
            break;
        default:
            break;
    }

    if (app_task_exitting()) {
        power_on_unint();
        return;
    }
}

```

IAL master apps/soundbox/task_manager/power_on/power_on.c

3.4 APP 模式管理函数接口介绍

APP 模式管理函数集中在 task_manager/app_task_switch.c

1.APP 模式配置表，这里可以配置切换模式的顺序，方案根据需求定义

```
///模式配置表，这里可以配置切换模式的顺序，方案根据需求定义
static const u8 app_task_list[] = {
#ifdef TCFG_APP_BT_EN
    APP_BT_TASK,
#endif
#ifdef TCFG_APP_MUSIC_EN
    APP_MUSIC_TASK,
#endif
#ifdef TCFG_APP_FM_EN
    APP_FM_TASK,
#endif
#ifdef TCFG_APP_RECORD_EN
    APP_RECORD_TASK,
#endif
#ifdef TCFG_APP_LINEIN_EN
    APP_LINEIN_TASK,
#endif
#ifdef TCFG_APP_RTC_EN
    APP_RTC_TASK,
#endif
#ifdef TCFG_APP_PC_EN
    APP_PC_TASK,
#endif
#ifdef TCFG_APP_SPDIF_EN
    APP_SPDIF_TASK,
#endif
};
```

2. 相关函数介绍

```
/*-----*/
/**@brief    模式退出检查
  @param    curr_task:当前模式
  @return    TRUE可以退出， FALSE不可以退出
  @note
*/
/*-----*/
static int app_task_switch_exit_check(u8 curr_task)
{
    int ret = false;
```

```
3 /*-----*/
4 /**@brief    模式进入检查
5  @param    app_task:目标模式
6  @return    TRUE可以进入， FALSE不可以进入
7  @note      例如一些需要设备在线的任务(music/record/linein等)，
8             如果设备在线可以进入， 没有设备在线不进入可以在这里处理
9  */
10 /*-----*/
11 static int app_task_switch_check(u8 app_task)
12 {
13     int ret = false;
14     switch (app_task) {
15 #ifdef TCFG_APP_MUSIC_EN
16     case APP_MUSIC_TASK:
```

```
/**-----*/
/**@brief    切换到上一个模式
  @param
  @return
  @note
*/
void app_task_switch_prev()
{
    int i = 0;
    int cur_index = 0;

    if (app_next_task) {
```

```
/**-----*/
/**@brief    切换到下一个模式
  @param
  @return
  @note
*/
void app_task_switch_next()
{
    int i = 0;
    int cur_index = 0;

    if (app_next_task) {
        printf("app_task_switch_next busy \n");
    }
```

```
/**-----*/
/**@brief    切换到指定模式
  @param    app_task:指定模式
  @return
  @note
*/
int app_task_switch_to(u8 app_task)
{
    if (app_curr_task == app_task) {
        return false;
    }

    if (!app_task_switch_check(app_task)) {
        return false;
    }
}
```

```
0
1 /**-----*/
2 /**@brief 跳回到原来的模式
3  @param
4  @return
5  @note
6  */
7 /**-----*/
8 int app_task_switch_back()
9 {
10     if (app_prev_task == 0) {
11         return -EINVAL;
12     }
13     return app_task_switch_to(app_prev_task);
14 }
```

```
/**-----*/
/**@brief 模式切换退出检测
 @param
 @return 1:响应退出模式, 0:不响应
 @note
 */
/*-----*/
u8 app_task_exitting()//
{
    struct sys_event clear_key_event = {.type = SYS_KEY_EVENT, .arg = (void *)DEVICE_EVENT_FROM_KEY};
    if (app_next_task != 0) {
        ! app_curr_task = app_next_task;
        ! app_prev_task = 0;
    }
}
```

```
/**-----*/
/**@brief 获取当前模式
 @param
 @return 当前模式id
 @note
 */
/*-----*/
u8 app_get_curr_task()
{
    return app_curr_task;
}
```

```
/**-----*/
/**@brief 通过指定id检查是否是当前模式
 @param
 @return true:是当前模式, false:不是当前模式
 @note
 */
/*-----*/
u8 app_check_curr_task(u8 app)
{
    if (app == app_curr_task) {
        ! return true;
    }
    return false;
}
```


3.5 增加 APP 模式例子（以 music 为例）

1. 在 app_task.h 中增加模式 id（以 music 为例）

```
9 enum {
10     APP_POWERON_TASK = 1,
11     APP_POWEROFF_TASK = 2,
12     APP_BT_TASK = 3,
13     APP_MUSIC_TASK = 4,
14     APP_FM_TASK = 5,
15     APP_RECORD_TASK = 6,
16     APP_LINEIN_TASK = 7,
17     APP_RTC_TASK = 8,
18     APP_PC_TASK = 9,
19     APP_SPDIF_TASK = 10,
20     APP_IDLE_TASK = 11,
21     APP_TASK_MAX_INDEX,
22 };
```

2. 将新加的模式 id 加入到 app_task_switch.c 中模式配置表 app_task_list

```
6 ///模式配置表，这里可以配置切换模式的顺序，方案根据需求定义
7 static const u8 app_task_list[] = {
8     #if TCFG_APP_BT_EN
9         APP_BT_TASK,
10    #endif
11    #if TCFG_APP_MUSIC_EN
12        APP_MUSIC_TASK,
13    #endif
14    #if TCFG_APP_FM_EN
15        APP_FM_TASK,
16    #endif
17    #if TCFG_APP_RECORD_EN
18        APP_RECORD_TASK,
19    #endif
20    #if TCFG_APP_LINEIN_EN
21        APP_LINEIN_TASK,
22    #endif
23    #if TCFG_APP_RTC_EN
24        APP_RTC_TASK,
25    #endif
26    #if TCFG_APP_PC_EN
27        APP_PC_TASK,
28    #endif
29    #if TCFG_APP_SPDIF_EN
30        APP_SPDIF_TASK,
31    #endif
32 };
```

3. 参考 task_key.c 中参考添加模式按键转换表（ad、io、ir 等）

4. 在 task_manager 中添加对应的模式目录（及对应的头文件目录）

5. 实现模式相关接口（参考已有模式，以下以 music 为例进行说明）

① 实现以下基础必要接口：

```
void app_music_task()
```

```
int music_app_check(void)
```

```
static int music_sys_event_handler(struct sys_event *event)
```

```
static int music_key_event_opr(struct sys_event *event)
```

② 模式主循环内完成以下基础操作（app_music_task）

获取消息

响应消息及事件

响应模式内部消息及事件

响应公共消息及事件

```
void app_music_task()
{
    int res;
    int msg[32];
    music_task_start(); // 初始化, 非必要, 根据具体情景定义

    int err = tone_play_with_callback_by_name(tone_table[INDEX_TONE_MUSIC], 1, music_tone_play_end_callback, (void *)INDEX_TONE_MUSIC);
    if (err) {
        music_player_play_start(); // 播放模式提示音, 非必要, 如果要的话参考music.c的案例实现
    }

    while (1) { // 模式主循环
        app_task_get_msg(msg, ARRAY_SIZE(msg), 1); // 获取消息
        switch (msg[0]) {
            case APP_MSG_SYS_EVENT: // 处理系统case消息
                if (music_sys_event_handler((struct sys_event *)&msg[1]) == false) { // 模式内部消息拦截, 如果不拦截, 给公共消息处理响应
                    app_default_event_deal((struct sys_event *)&msg[1]); // 必要, 响应公共消息处理
                }
                break;
            default:
                break;
        }
        if (app_task_exitting()) { // 必要, 模式退出检测处理
            music_task_close(); // 模式退出内部释放处理, 非必要, 如果有初始化, 就一定要注意释放操作
            return;
        }
    }
}
```

③ 在 app_main.c 中调用对应的模式主循环接口（app_music_task）

```
38 void app_task_loop()
39 {
40     while (1) {
41         switch (app_curr_task) {
42             case APP_POWERON_TASK:
43                 log_info("APP_POWERON_TASK \n");
44                 app_poweron_task();
45                 break;
46             case APP_POWEROFF_TASK:
47                 log_info("APP_POWEROFF_TASK \n");
48                 app_poweroff_task();
49                 break;
50             case APP_BT_TASK:
51                 log_info("APP_BT_TASK \n");
52                 app_bt_task();
53                 break;
54             case APP_MUSIC_TASK:
55                 log_info("APP_MUSIC_TASK \n");
56                 app_music_task();
57                 break;
58             case APP_FM_TASK:
59                 log_info("APP_FM_TASK \n");
60                 app_fm_task();
61                 break;
62             case APP_RECORD_TASK:
63                 log_info("APP_RECORD_TASK \n");
64                 app_record_task();
65                 break;
66         }
67     }
68 }
```

NORMAL ▶ master ▶ apps/soundbox/app_main.c

④ app_check 接口的实现（music 为例）

app_check 其实是在切换模式的时候，是否满足条件进入该模式，music 模式进入条件是判断是否有可以播放的设备在线，故接口实现如下：

```
634 int music_app_check(void)
635 {
636     if (dev_manager_get_total(1)) {
637         return true;
638     }
639     return false;
640 }
```

⑤ 在 app_task_switch_check 调用 app_check（music 为例）

```
107 static int app_task_switch_check(u8 app_task)
108 {
109     int ret = false;
110     switch (app_task) {
111 #if TCFG_APP_MUSIC_EN
112     case APP_MUSIC_TASK:
113         ret = music_app_check();
114         break;
115 #endif
116 #if TCFG_APP_LINEIN_EN
117     case APP_LINEIN_TASK:
118         ret = linein_app_check();
119         break;
120 #endif
121 #if TCFG_APP_PC_EN
```

Chapter 4 APP 消息管理

4.1 系统事件管理

1、系统事件的分类

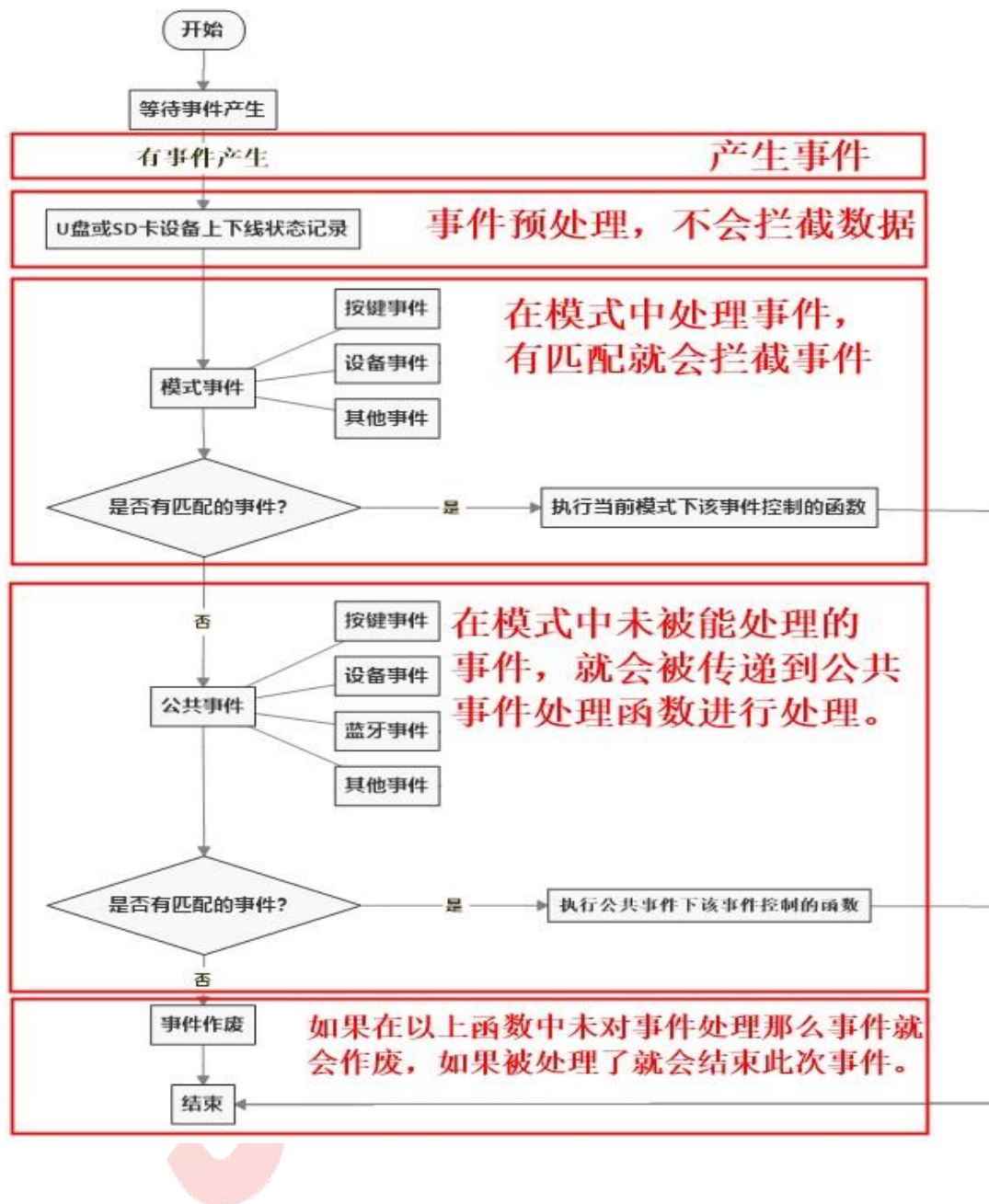
系统事件分为系统按键事件、系统设备事件以及系统蓝牙事件，为了更好管理，还把这三大事件分为模式里的系统事件类和每个模式都有的公共系统事件类。流程图如下：

特别说明：

①系统产生事件后，会先到模式里的系统事件查询是否有匹配的事件，如有，则执行相应的函数并且不再执行公共系统事件类，然后结束事件，如无，则到公共系统事件查询是否有匹配的事件，如有，则执行相应的函数，然后结束事件，如无，则视为事件作废，结束事件。

②蓝牙模式的设备事件如果不做处理，那么会把事件丢到公共事件中的公共蓝牙事件去处理，而不是丢到公共事件的公共设备事件去处理。

以下为系统事件的管理机制：



4.2 提供的 APP 消息处理函数说明

1.提供的消息函数接口

```
5
6
7 //app自定义消息发送接口
8 int app_task_put_usr_msg(int msg, int arg_num, ...);
9
10 //app消息获取接口(block参数为0表示内部pend, 1直接返回)
11 void app_task_get_msg(int *msg, int msg_size, int block);
12
13 //app按键消息发送接口
14 int app_task_put_key_msg(int msg, int value);
15
16 //app清理按键消息接口
17 void app_task_clear_key_msg();
18
```

2.消息处理机制

(1) 获取消息

```
1 void app_poweron_task()
2 {
3     int msg[32];
4
5     UI_SHOW_MENU(MENU_POWER_UP, 0, 0, NULL);
6
7     int err = tone_play_with_callback_by_name(tone_table[IDEX_TONE_POWER_ON], 1, tone_play_end_callback, (void *)IDEX_TONE_POWER_ON);
8     if (err) { //提示音没有,播放失败, 直接init流程
9         power_on_init();
10    }
11
12    while (1) {
13        app_task_get_msg(msg, ARRAY_SIZE(msg), 1);
14        switch (msg[0]) {
15            case APP_MSG_SYS_EVENT:
16                if (poweron_sys_event_handler((struct sys_event *) (msg + 1)) == false) {
17                    app_default_event_deal((struct sys_event *) (&msg[1])); //由common统一处理
18                }
19                break;
20            default:
21                break;
22        }
23
24        if (app_task_exitting()) {
25            power_on_unint();
26            return;
27        }
28    }
29 }
```

(2) 推出 app 按键处理消息（非物理按键触发）

```

        break;
    }
    if (logo && (0 == memcmp(logo, evt_logo, str_len))) {
        //相同的设备才响应
        if (music_player_get_playing_breakpoint(breakpoint, 1) == true) {
            breakpoint_vm_write(breakpoint, logo);
        }
        //停止解码,防止设备掉线后还继续使用
        music_player_stop(1);
        //重新选择活动设备播放
        app_task_put_key_msg(KEY_MUSIC_PLAYER_START, 0); //卸载了设备再执行
        log_i("KEY_MUSIC_PLAYER_START_AFTER_UMOUNT\n");
    }
} else {
    //新设备上线,先记录当前设备断点,然后播放活动设备
    if (music_player_get_playing_breakpoint(breakpoint, 1) == true) {
        breakpoint_vm_write(breakpoint, logo);
    }
    //停止解码,播放新活动设备
    music_player_stop(1);
    app_task_put_key_msg(KEY_MUSIC_PLAYER_START, 0);
    log_i("KEY_MUSIC_PLAYER_START_AFTER_MOUNT\n");
    //先挂载了设备再执行
}
break;
default://switch((u32)event->arg)
break;

```

3. app 自定义消息发送接口

(1) 消息值定义

```

4
5 enum {
6     APP_MSG_SYS_EVENT = Q_EVENT,
7
8     /* 用户自定义消息 */
9     APP_MSG_SWITCH_TASK = Q_USER + 1,
10    APP_MSG_USER          = Q_USER + 2,
11
12    // 添加消息值
13    // eg:
14    // APP_MSG_TEST      = Q_USER + 3,
15    //
16
17 },
18
19
20 //切换到前一个有效模式
21 void app_task_switch_prev();
22 //切换到下一个有效模式
23

```

(2) 消息处理

以蓝牙模式为例（如果消息其他模式都需要处理，就要在需要处理的模式都添加）

```
void app_bt_task()
{
    int res;
    int msg[32];
    ui_update_status(STATUS_EXIT_LOWPOWER);

    if (!__this->cmd_flag) { //蓝牙后台拉回蓝牙模式不播放提示音
        tone_play_by_path(tone_table[IDEX_TONE_BT_MODE], 1);
    }

    bt_task_start();

    while (1) {
        app_task_get_msg(msg, ARRAY_SIZE(msg), 1);

        switch (msg[0]) {
            case APP_MSG_SYS_EVENT:
                if (bt_sys_event_handler((struct sys_event *) (msg + 1)) == false) {
                    app_default_event_deal((struct sys_event *) (&msg[1]));
                }
                break;

            /* case APP_MSG_TEST:
                // deal */
            /* break; */

            default:
                break;
        }

        if (app_task_exitting()) {
            bt_task_close();
            __this->wait_exit = 1;
        }

        if (__this->wait_exit) { //开始退出
            if (!__this->exiting) { ///等待蓝牙退出处理完成
                return;
            }
        }
    }
}
```


Chapter 5 SD、USB 复用 IO 介绍

5.1 编写目的

该章节主要描述 JL701N、AC696N 系列芯片 sd cmd 复用 linein ad 检测、sd clk 复用 iokey，sd data 与 usb dm 绑定复用的介绍说明。**(sd cmd 复用 linein ad 检测、sd clk 复用 iokey 需要硬件电路支持, 硬件电路的接法不在本文讨论, 在制定方案时候请咨询硬件)**

5.2 sd 复用 api 说明

函数原型	u8 sd_io_suspend(u8 sdx, u8 sd_io);
功能描述	强占 sd io 资源
参数说明	sdx 0:sd0 1:sd1 sd_io 0:cmd 1:clk 2:data
输出	0:强占成功 1: 强占失败, 需要 sd 当前正在使用, 需要再次强占
例子	<pre> __try_t: if (sd_io_suspend(0, 0)) { os_time_dly(1); printf("wait %s\n", __FUNCTION__); goto __try_t; } </pre>
补充说明	cmd 和 data 使用了同一个控制器, 如果有复用 data io 的应用, 则不能使用 cmd 检测的方式

函数原型	u8 sd_io_resume(u8 sdx, u8 sd_io)
------	-----------------------------------

功能描述	释放强占 sd io 资源
参数说明	Sdx 0:sd0 1:sd1 sd_io 0:cmd 1:clk 2:data
输出	0:释放强占成功 1: 释放强占失败
例子	sd_io_resume(0, 0);
补充说明	强占操作和释放操作必须成对出现，如果有强占没有释放会影响 sd 通信

5.3 sd cmd 复用 linein ad 检测

```

5
6 //*****
7 //***** linein配置 *****//
8 //*****
9 //*****
10 #define TCFG_LINEIN_ENABLE          TCFG_APP_LINEIN_EN // linein使能
11 // #define TCFG_LINEIN_LADC_IDX      0 // linein使用的ladc通道，对应ladc_list
12 #define TCFG_LINEIN_LR_CH           AUDIO_LR           // linein检测IO
13 #define TCFG_LINEIN_CHECK_PORT       IO_PORTC_04        // 检测IO上拉使能
14 #define TCFG_LINEIN_PORT_UP_ENABLE   1 // 检测IO下拉使能
15 #define TCFG_LINEIN_PORT_DOWN_ENABLE 0 // 检测IO是否使用AD检测
16 #define TCFG_LINEIN_AD_CHANNEL       AD_CH_PC4//NO_CONFIG_PORT // AD检测时的阈值
17 #define TCFG_LINEIN_VOLTAGE          1000 // linein 检测与 SD cmd 复用
18 #define TCFG_LINEIN_INPUT_WAY        LINEIN_INPUT_WAY_ADC
19 #define TCFG_LINEIN_MULTIPLEX_WITH_SD ENABLE //选择复用的sd
20 #define TCFG_LINEIN_SD_PORT          1// 0:sd0 1:sd1
21

```

一个典型的 linein 复用 sd cmd 配置如上图所示，下面对参与复用的配置项目进行描述。

TCFG_LINEIN_CHECK_PORT：为 ad 的检测 io，需要填写为 sd cmd 的具体 io，根据具体方案来进行填写。

TCFG_LINEIN_AD_CHANNEL：为 ad 的通道，头文件在 adc_api.h，用户可以根据具体方案填写，本 demo 为 AD_CH_PC4。

TCFG_LINEIN_VOLTAGE：为 ad 检测的阈值，用户可以在 linein_sample_mult_sd 函数把 ad 值进行打印出来，判断阈值。SDK 判断当前 ad 值比阈值低时候，设备判断为插入。

TCFG_LINEIN_MULTIPLEX_WITH_SD : 为复用的开关。

TCFG_LINEIN_SD_PORT : 为 sd 的 port, 需要根据具体方案进行调整, 例如是 sd0 的 cmd 参与复用, 则此处填写 0, 反之如果是 sd1 的 cmd 参与复用, 则此处是填写 1。

5.4 sd clk 复用 iokey 检测

```
49
50
51 #define TCFG_IO_MULTIPLEX_WITH_SD    ENABLE           // IO_KEY与 SD_CLK复用
52 #define TCFG_MULTIPLEX_PORT         TCFG_IOKEY_PREV_ONE_PORT // 复用的引脚
53
```

如图, 在板卡中增加对应的配置即可支持 sd clk 复用 iokey 的检测, clk io 和按键之间需要串联一个小电阻, 细节的硬件电路的接法不在本文讨论。

TCFG_IO_MULTIPLEX_WITH_SD : 为复用开关。

TCFG_MULTIPLEX_PORT : 为复用的引脚 io, 这里填写具有方案的 clk 引脚。注意事项, 因为具体应用方案可能是 sd0、sd1 的 clk io 参与了复用, 因此在代码中需要注意修改参与复用的 sd。如下图函数在 io_key.c 的 sd_mult_io_detect 函数

```
133 write ((JL_TIMER0->CON & BIT(15)) == 0),
134 JL_TIMER0->CON = BIT(14);
135 }
136 extern u8 sd_io_suspend(u8 sdx, u8 sd_io);
137 extern u8 sd_io_resume(u8 sdx, u8 sd_io);
138 void sd_mult_io_detect(void *arg)
139 {
140     static u32 cnt = 0;
141     if (sd_io_suspend(1, 1) == 0) {
142         gpio_set_direction(TCFG_MULTIPLEX_PORT, 0);
143         gpio_write(TCFG_MULTIPLEX_PORT, 0);
144         udelay(10);
145         gpio_set_dir(TCFG_MULTIPLEX_PORT, 1);
146         gpio_set_pull_down(TCFG_MULTIPLEX_PORT, 0);
147         gpio_set_pull_up(TCFG_MULTIPLEX_PORT, 1);
148         gpio_set_direction(TCFG_MULTIPLEX_PORT, 1);
149         udelay(10);
150         mult_key_value = gpio_read(TCFG_MULTIPLEX_PORT);
151         sd_io_resume(1, 1);
152     }
153 }
154 #endif
155
```

修改对应sd

5.5 sd data 复用 usb dm

如下图, 一个典型的 sd data 复用配置

```

37 //***** USB 配置 *****//
38 //
39 //*****
40 #define TCFG_PC_ENABLE          TCFG_APP_PC_EN//PC模块使能
41 #define TCFG_UDISK_ENABLE      ENABLE_THIS_MODULE//U盘模块使能
42 #define TCFG_OTG_USB_DEV_EN    BIT(0)//USB0 = BIT(0)  USB1 = BIT(1)
43
44 #include "usb_std_class_def.h"
45
46 #define TCFG_USB_PORT_CHARGE    DISABLE
47
48
49 #define TCFG_USB_DM_MULTIPLEX_WITH_SD_DAT0    1
50 #if TCFG_USB_DM_MULTIPLEX_WITH_SD_DAT0
51 //复用情况下，如果使用此USB口作为充电（即LDO5V_IN连接到此USB口），
52 //TCFG_OTG_MODE需要或上TCFG_OTG_MODE_CHARGE，用来把charge从host区
53 //分开；否则不需要，如果LDO5V_IN与其他IO绑定，则不能或上
54 #define TCFG_DM_MULTIPLEX_WITH_SD_PORT    0//0:sd0  1:sd1 //dm 参与复用的sd配置
55 #undef TCFG_OTG_MODE
56 #define TCFG_OTG_MODE                    (TCFG_OTG_MODE_HOST|TCFG_OTG_MODE_SLAVE|TCFG_OTG_MODE_CHARGE|OTG_DET_DP_ONLY)
57
58 #undef USB_DEVICE_CLASS_CONFIG
59 #if TCFG_SD0_SD1_USE_THE_SAME_HW //开启了双卡的可以使能读卡器存储设备
60 #define USB_DEVICE_CLASS_CONFIG (MASSSTORAGE_CLASS|SPEAKER_CLASS|MIC_CLASS|HID_CLASS)
61 #else
62 #define USB_DEVICE_CLASS_CONFIG (SPEAKER_CLASS|MIC_CLASS|HID_CLASS)
63 #endif
64
65 #undef TCFG_SD0_DET_MODE
66 #define TCFG_SD0_DET_MODE            SD_CLK_DECT
67 #define TCFG_USB_SD_MULTIPLEX_IO    IO_PORTB_03
68
69 #endif
70

```

下面对参与 usb 和 sd 复用的配置项目进行说明：

TCFG_USB_DM_MULTIPLEX_WITH_SD_DAT0 ：复用的总开关。

TCFG_DM_MULTIPLEX_WITH_SD_PORT :参与复用的 sd, 根据具体的封装应用场景进行填写。0 为 sd0、1 为 sd1。

TCFG_OTG_MODE :复用情况下，如果使用此 USB 口作为充电（即 LDO5V_IN 连接到此 USB 口），TCFG_OTG_MODE 需要或上 TCFG_OTG_MODE_CHARGE，用来把 charge 从 host 区分开；否则不需要，如果 LDO5V_IN 与其他 IO 绑定，则不能或上。

USB_DEVICE_CLASS_CONFIG :从机模式支持的功能配置，如果只有单卡的场景，usb 和 sd 复用下，不支持读卡器功能。注意事项：在插入 usb 线场景下播放 sd 卡音乐，sd 卡播放可能有影响，这是无法解决的，干扰来源来着电脑端。用户可以选择性关闭 pc 模式。

TCFG_USB_SD_MULTIPLEX_IO ：为 sd 卡 data io,用户需要根据具体的应用方案进行填写。

注意事项：如下图 usb 和 sd data 复用场景 sd 不能使用 cmd 检测，只是 sd 控制器决定，因此本 demo 默认重新修改了检测方式为 clk 检测。用户需要根据具体的方案应用选择 clk 检测或者 io 检测。特别注意，如果是 sd1 参与复用，下面的检测方式应该是修改为 sd1 的对应配置。

```

163 #endif
164
165 #undef TCFG_SD0_DET_MODE
166 #define TCFG_SD0_DET_MODE    SD_CLK_DECT
167 #define TCFG_USB_SD_MULTIPLEX_IO    IO_PORTB_03
168

```

Chapter 6 线程优先级和时钟

6.1 线程优先级设置

文件: task_table.c

关键数组: `const struct task_info task_info_table[]`

```
5 /*任务列表, 注意:stack_size设置为32*n*/
6 const struct task_info task_info_table[] = {
7     {"app_core",      1,    0,   896,   768 },
8     {"sys_event",     6,    0,   256,    0 },
9     {"btctrler",      4,    0,   512,   384 },
10    {"btencry",        1,    0,   512,   128 },
11    #if TCFG_USER_TWS_ENABLE || TCFG_VIRTUAL_FAST_CONNECT_FOR_EMITTER
12    {"tws",            5,    0,   512,   128 },
13    #endif
14    #if (TCFG_USER_TWS_ENABLE || TCFG_USER_BLE_ENABLE || TCFG_USER_EMITTER_ENABLE)
15    {"btstack",        3,    0,   768,   256 },
16    #else
17    {"btstack",        3,    0,   512,   256 },
18    #endif
19    #if TCFG_DEC2TWS_TASK_ENABLE
20    {"local_dec",      3,    1,   768,   128 },
21    #endif
22    #if ((TCFG_USER_TWS_ENABLE && TCFG_MIC_EFFECT_ENABLE) || (SOUNDCARD_ENABLE))
23    {"audio_dec",      3,    1,  768 + 128 + 128,  128 },
24    #else
25    {"audio_dec",      3,    1,  768 + 32,   128 },
26    #endif
27    {"audio_enc",      3,    0,   512,   128 },
28    #if TCFG_AUDIO_DEC_OUT_TASK
29    {"audio_out",      2,    0,   384,    0 },
30    #endif
31    {"aec",            2,    0,   768,    0 },
32    {"aec_dbg",        3,    0,   128,   128 },
33    {"update",         1,    0,   512,    0 },
34    #if (USER_UART_UPDATE_ENABLE)
35    {"uart_update",    1,    0,   256,   128 },
36    #endif
37    {"sys timer",      6,    0,   128,    0 },
38    {"dev mg",         3,    0,   512,   512 },
39    #ifndef CONFIG_SOUNDBOX_FLASH_256K
40    {"usb_msd",        1,    0,   512,   128 },
41    {"usb_audio",      5,    0,   256,   256 },
42    {"plnk_dbg",       5,    0,   256,   256 },
43    {"adc_linein",     2,    0,   768,   128 },
44    /* {"src_write",    1,    0,   768,   256 }, */
45 }
46 #endif
```

→ 优先级

说明: 最低优先级: 0; 最高优先级: 6; 没有特殊要求不建议修改, 避免引起系统崩溃。

6.2 时钟设置

文件: clock_manager.c

关键数组: const struct clock_type clock_enum[]

```
const struct clock_type clock_enum[] = {  
    { BT_IDLE_CLOCK      , (24), " BT_IDLE_CLOCK      " },  
    { MUSIC_IDLE_CLOCK   , (24), " MUSIC_IDLE_CLOCK   " },  
    { FM_IDLE_CLOCK      , (48), " FM_IDLE_CLOCK      " },  
    { LINEIN_IDLE_CLOCK  , (96), " LINEIN_IDLE_CLOCK  " },  
    { PC_IDLE_CLOCK      , (120), " PC_IDLE_CLOCK      " },  
    { REC_IDLE_CLOCK     , (12), " REC_IDLE_CLOCK     " },  
    { RTC_IDLE_CLOCK     , (12), " RTC_IDLE_CLOCK     " },  
    { SPDIF_IDLE_CLOCK   , (12), " SPDIF_IDLE_CLOCK   " },  
    { BOX_IDLE_CLOCK     , (24), " BOX_IDLE_CLOCK     " },  
    { DEC_SBC_CLK        , (32), "DEC_SBC_CLK        " },  
    { DEC_AAC_CLK        , (32), "DEC_AAC_CLK        " },  
    { DEC_MSBC_CLK       , (12), "DEC_MSBC_CLK       " },  
    { DEC_CVSD_CLK       , (8), "DEC_CVSD_CLK       " },  
    { AEC8K_CLK          , (48), "AEC8K_CLK          " },  
    { AEC8K_ADV_CLK      , (60), "AEC8K_ADV_CLK      " },  
    { AEC16K_CLK         , (80), "AEC16K_CLK         " },  
    { AEC16K_ADV_CLK     , (120), "AEC16K_ADV_CLK     " },  
    { DEC_TONE_CLK       , (24 + 32), "DEC_TONE_CLK       " },  
    { DEC_MP3_CLK        , (46), "DEC_MP3_CLK        " },  
    { DEC_WAV_CLK        , (24), "DEC_WAV_CLK        " },  
    { DEC_G729_CLK       , (24), "DEC_G729_CLK       " },  
    { DEC_G726_CLK       , (24), "DEC_G726_CLK       " },  
}
```

红色框为每个模式的 idle 时钟，在进入每一个模式会配置一次；

后面的时钟是每一种算法或者运行的操作需要添加的时钟值，在要运行对应算法的时候需要添加对应时钟，在结束对应处理的时候要把时钟删除。

把时钟设置加入到 ext 中,但是不是立刻设置时钟,需要最后调用 clock_set_cur 来最后设置时钟,用于连续地方设置时钟,用完后必须把时钟 remove

```
void clock_add(u32 type)
```

```
void clock_remove(u32 type)
```

```
void clock_set_cur(void)
```

把时钟设置加入到 ext 中,立刻设置时钟,需要立刻添加时钟,用完后必须把时钟 remove

```
void clock_add_set(u32 type)
```

```
void clock_remove_set(u32 type)
```

如果想要固定频率,可以设置下面宏定义,为非 0 就是固定频率值

```
//// 如果clock_fix 为0 就按照配置设置时钟,如果有值就固定频率  
#define CLOCK_FIX 0//192
```

Chapter 7 Audio 模块使用

7.1 编写目的

该文档主要描述 JL701N 系列 Audio 模块的使用方法及开发中注意的一些问题，为用户进行二次开发提供参考。

7.2 在线 EQ 调试

1 软件上修改流程方法:

例如使用 AC897N 的 board_ac8976d_tws_cfg.c 配置。在 board_ac8976d_tws_cfg.h 打开如下设置。

```

//***** EQ配置 *****//
//EQ配置，使用在线EQ时，EQ文件和EQ模式无效。有EQ文件时，默认不用EQ模式切换功能
#define TCFG_EQ_ENABLE 1 //支持EQ功能,EQ总使能
#if TCFG_EQ_ENABLE
#define TCFG_EQ_ONLINE_ENABLE 1 //支持在线EQ调试,如果使用蓝牙串口调试,需要打开宏 APP_ONLINE_DEBUG,否则,默认使用uart调试(二选一)
#define TCFG_BT_MUSIC_EQ_ENABLE 1 //支持蓝牙音乐EQ
#define TCFG_PHONE_EQ_ENABLE 1 //支持通话近端EQ
#define TCFG_AUDIO_OUT_EQ_ENABLE 0 //mix out级,增加eq高低音接口
#define TCFG_AEC_DCCS_EQ_ENABLE 1 // AEC DCCS
#define TCFG_AEC_UL_EQ_ENABLE 1 // AEC UL
#define TCFG_USE_EQ_FILE 0 //是否使能默认系数表 1: 不使能 0:使能
#define EQ_SECTION_MAX 10 //eq 段数,最大20
#endif//TCFG_EQ_ENABLE

/*省电容mic通过eq模块实现去直流滤波*/
#if (TCFG_SUPPORT_MIC_CAPLESS && (TCFG_MIC_CAPLESS_ENABLE || TCFG_MIC1_CAPLESS_ENABLE))
#if ((TCFG_EQ_ENABLE == 0) || (TCFG_AEC_DCCS_EQ_ENABLE == 0))
#error "MicCapless enable,Please enable TCFG_EQ_ENABLE and TCFG_AEC_DCCS_EQ_ENABLE"
#endif
#endif
#endif
#if TCFG_EQ_ONLINE_ENABLE && TCFG_TEST_BOX_ENABLE
#error "在线调试时,请关闭TCFG_TEST_BOX_ENABLE"
#endif

```

1.2 通讯方式

使用串口通讯，默认使用 usb DP DM 做调试口，波特率 115200，可根据实际使用改用别的 IO

```

// ONLINE CONFIG
// 如果调试串口是DP DM,使用eq调试串口时,需关闭usb宏
#define TCFG_ONLINE_ENABLE (TCFG_EQ_ONLINE_ENABLE) //是否支持EQ在线调试功能
#define TCFG_ONLINE_TX_PORT IO_PORT_DP //EQ调试TX口选择
#define TCFG_ONLINE_RX_PORT IO_PORT_DM //EQ调试RX口选择

```

在线调试时，请关闭 TCFG_TEST_BOX_ENABLE，以免串口被占用，导致通讯失败

```

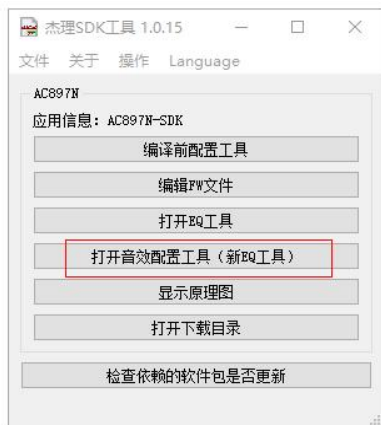
//***** 充电仓配置 *****//
//***** 充电仓配置 *****//
#define TCFG_CHARGESTORE_ENABLE ENABLE_THIS_MODULE //是否支持智能充电仓
#define TCFG_TEST_BOX_ENABLE 0 //是否支持ANC测试盒
#define TCFG_AEC_BOX_ENABLE 0 //是否支持ANC测试盒
#define TCFG_CHARGESTORE_PORT IO_PORTP_00 //耳机和充电仓通讯的IO口
#define TCFG_CHARGESTORE_UART_ID IRQ_UART1_IDX //通讯使用的串口号

```

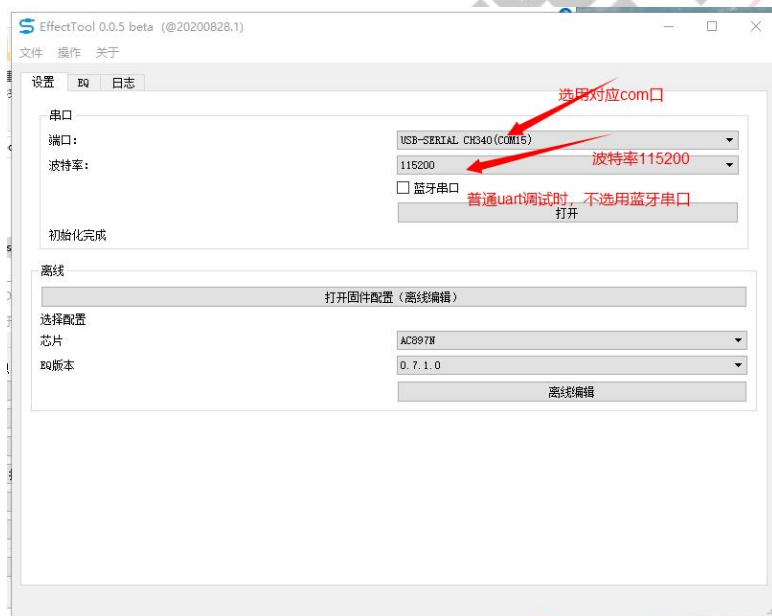
设置后编译，下载到开发板或者样机就可以了。

2 PC 端设置流程。

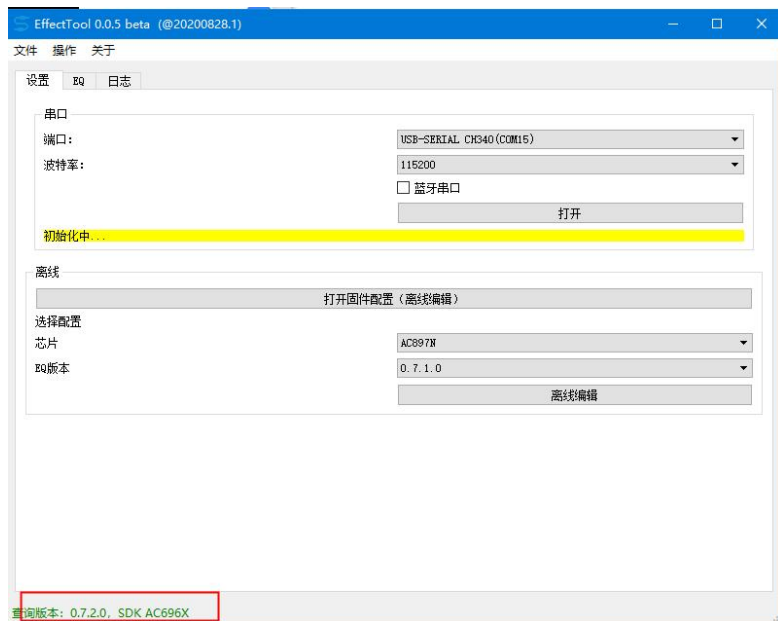
2.1 使用”杰理包管理器”软件打开，如下界面。选中新 EQ 工具



2.2 选用对应 COM 口，波特率 115200，普通 uart 调试时，不选蓝牙串口



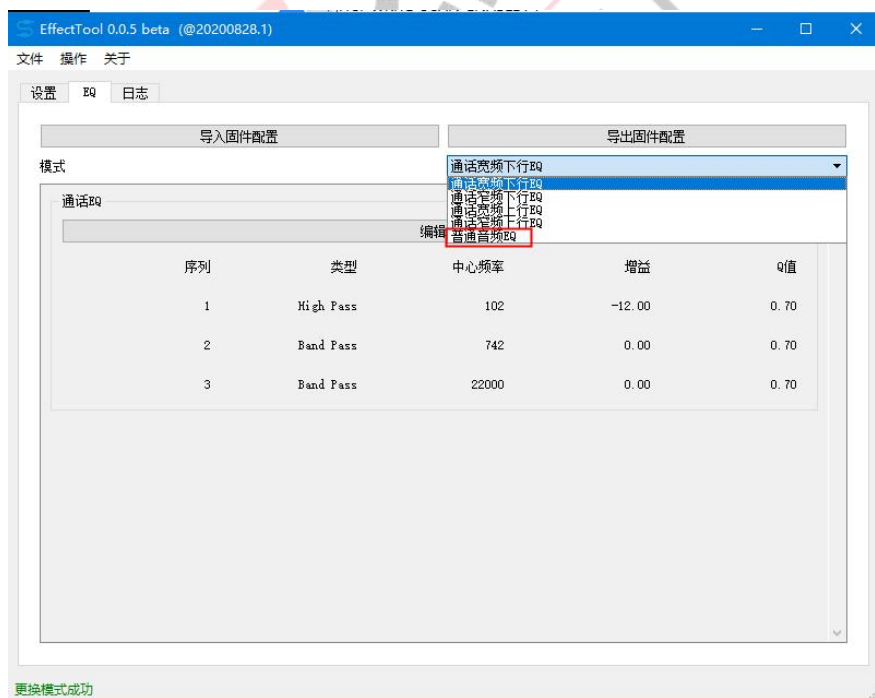
2.3 点击打开，通讯成功左下角会提示 查询版本



2.4 选中下拉列表，调试对应模式

下拉列表有：通话宽频下行 EQ、通话窄频下行 EQ、通话宽频上行 EQ、通话窄频下行 EQ、普通音频 EQ，五种模式。其中蓝牙、music、linein、fm、pc 模式播歌的 eq 对应普通音频 EQ。

例如：选中普通音频 EQ 后，点击编辑





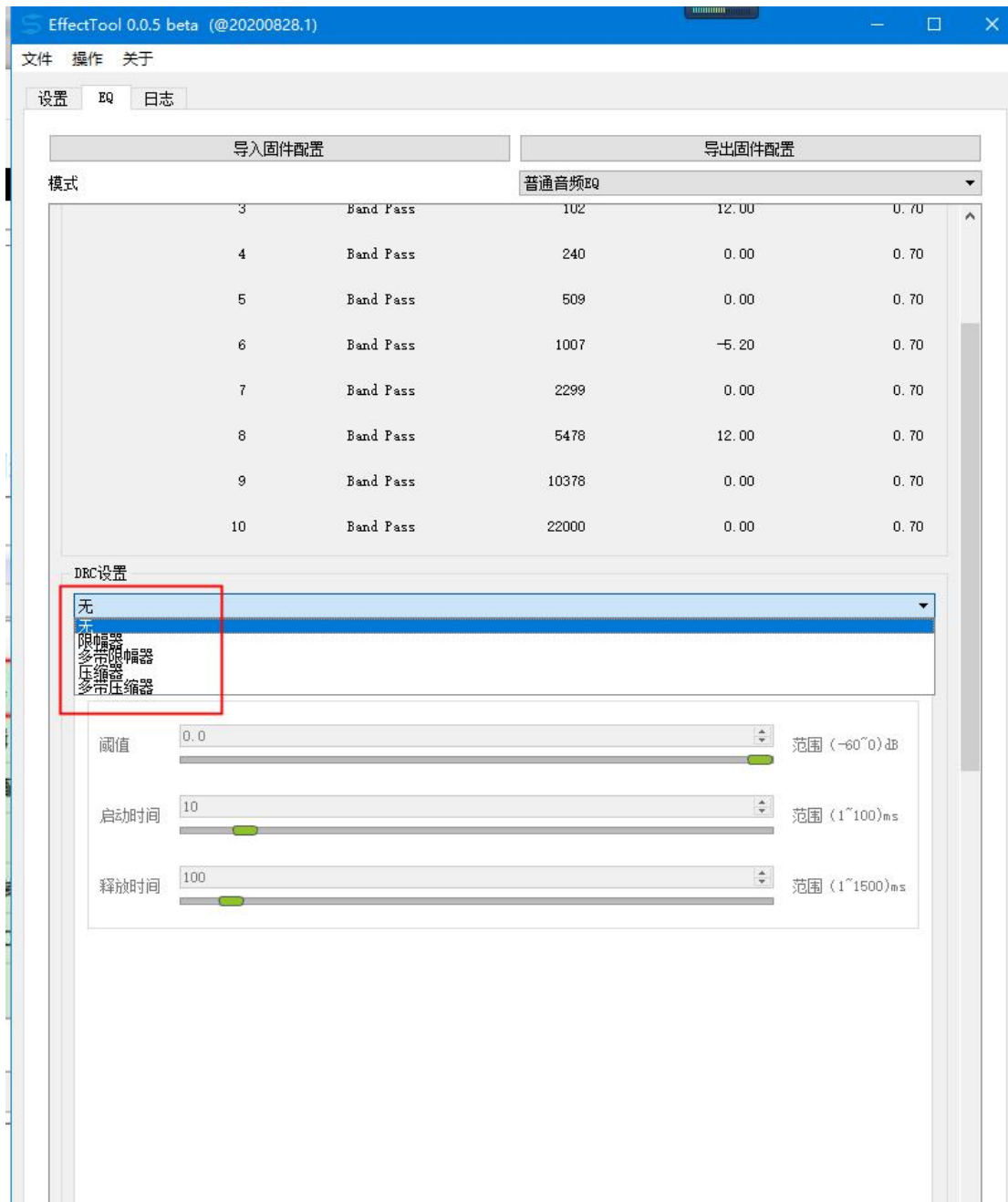


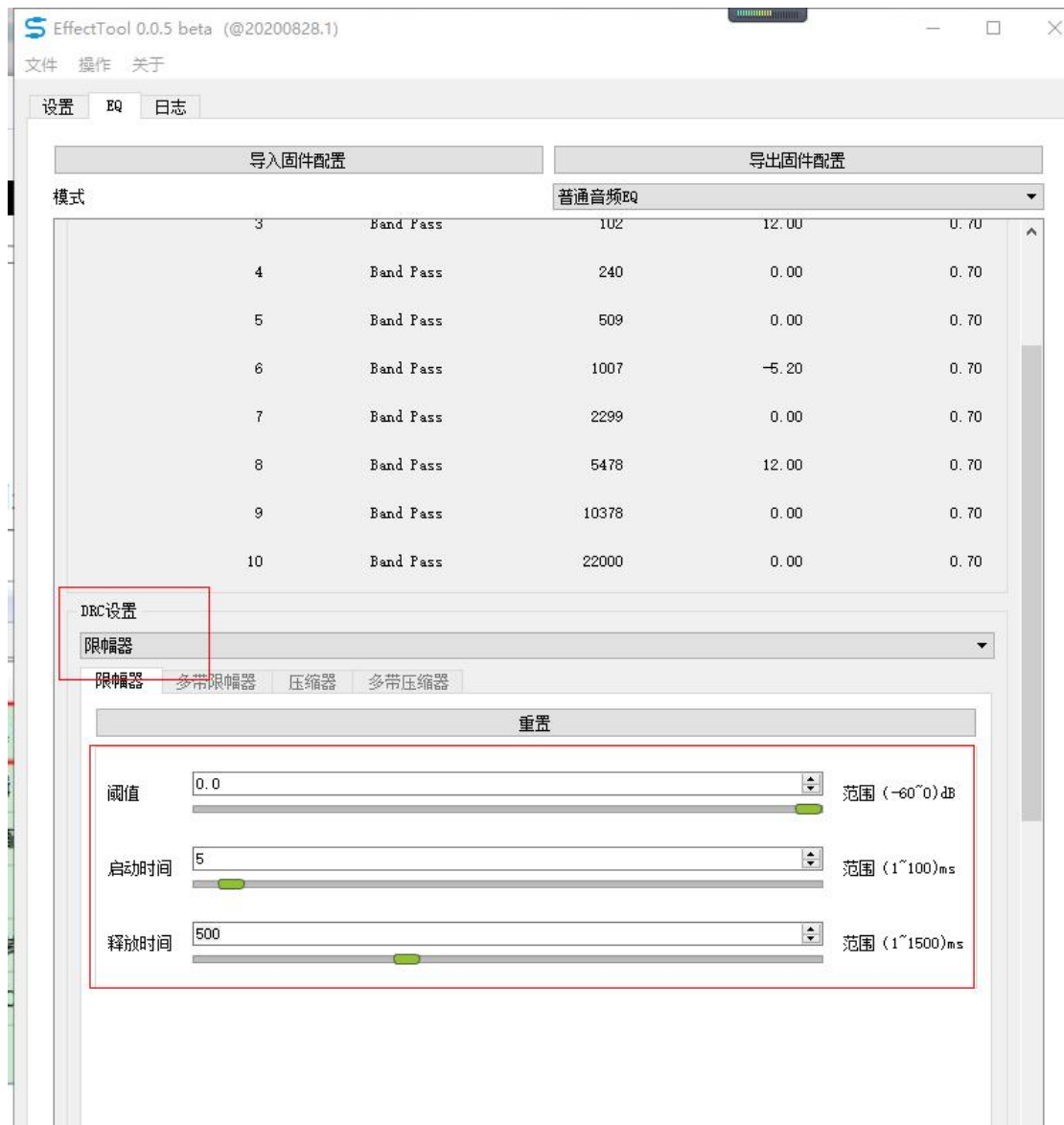
看到此界面，可以根据需对每一段 eq 的频率、增益、q 值以及滤波器类型进行调节，调节完毕点击 关闭



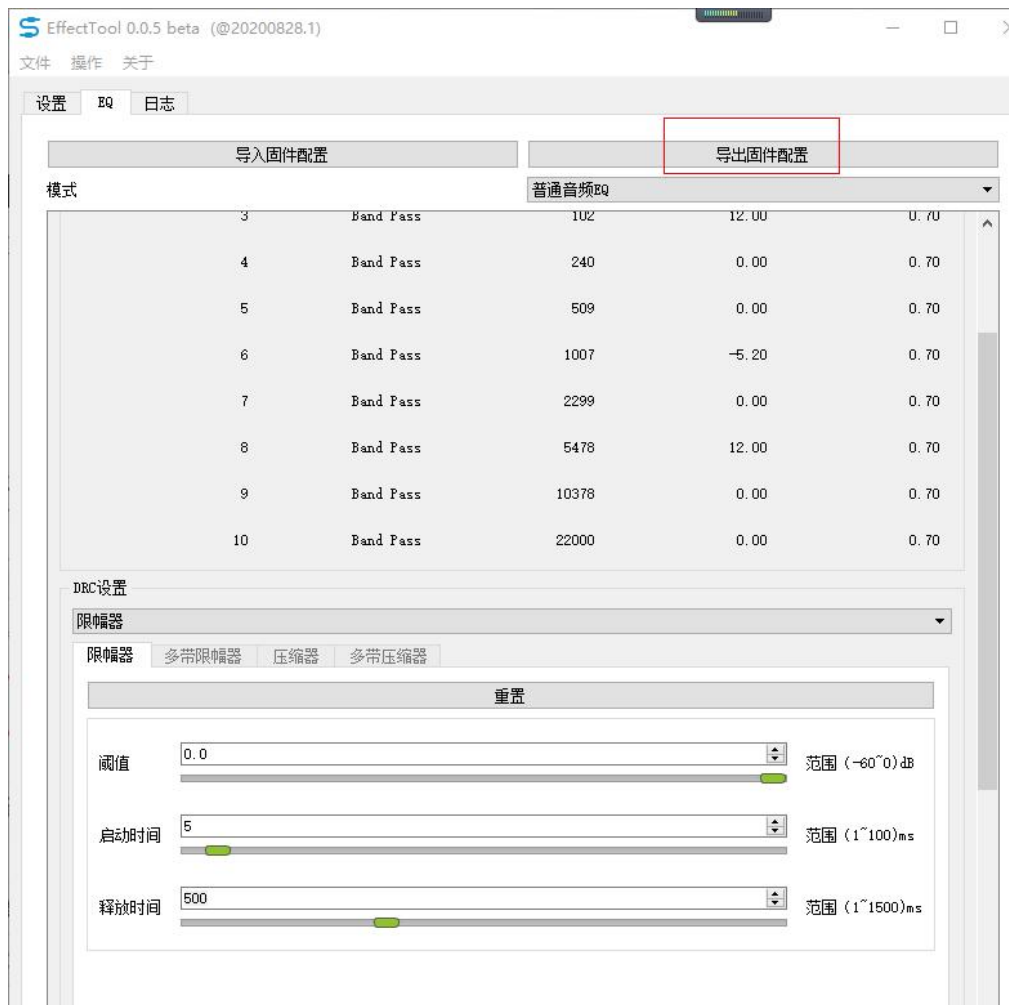
2.5 drc 限幅器、多带限幅器、压缩器、多带压缩器调节

选中 DRC 设置的下拉列表，例如选限幅器，进行设置。





2.6 EQ 与 DRC 调试完成后，点击导出固件 eq_cfg_hw.bin 文件。



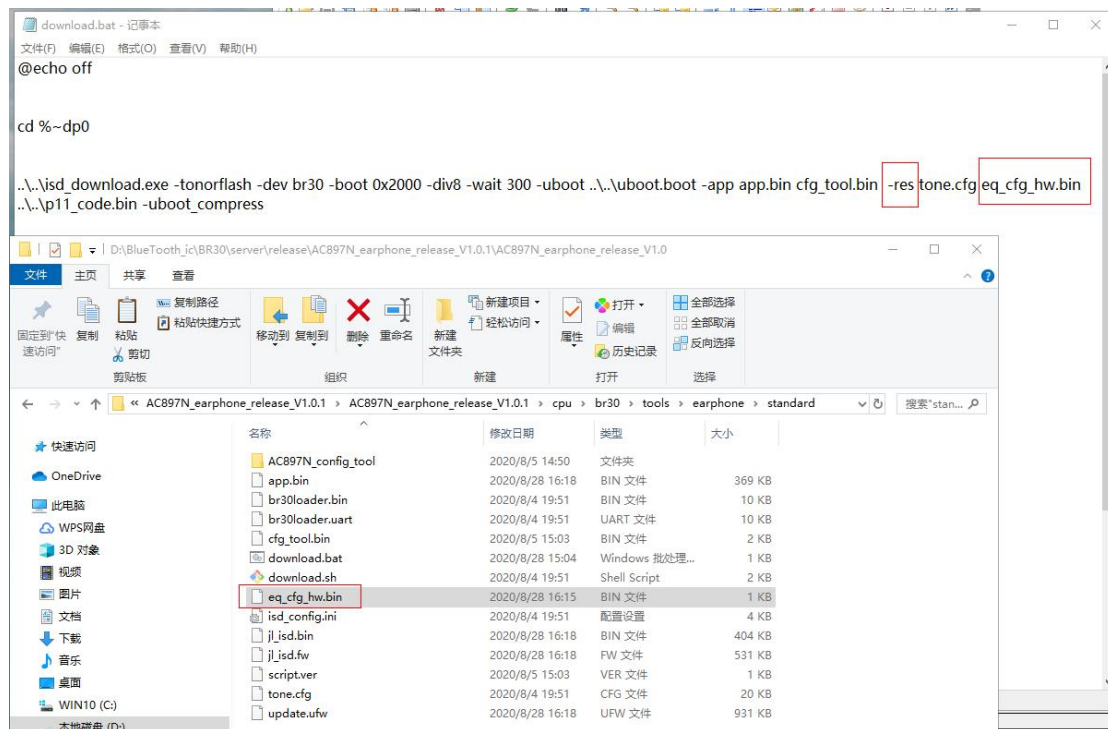
2.7 将 eq_hw_cfg.bin 文件放到下载目录，并在 download.bat 中-res 后添加 eq_hw_cfg.bin 文件。此外，调试完成需要将在线 eq 宏定义关闭,同时使能 使用 eq 文件宏定义

```

10
11 //*****EQ配置*****//
12 //*****EQ配置*****//
13 //*****EQ配置*****//
14 //EQ配置后，使用在线EQ时，EQ文件和EQ模式无效。有EQ文件时，默认不用EQ模式切换功能
15 #define TCFG_EQ_ENABLE 1 //支持EQ功能
16 #define TCFG_EQ_ONLINE_ENABLE 0 //支持在线EQ调试，如果使用蓝牙串口调试，需要打开宏 APP_ONLINE_DEBUG，否则，默认使用uart调试(二选一)
17 #define TCFG_BT_MUSIC_EQ_ENABLE 1 //支持蓝牙音乐EQ
18 #define TCFG_PHONE_EQ_ENABLE 1 //支持电话近端EQ
19 #define TCFG_AUDIO_OUT_EQ_ENABLE 0 //mix out级，增加eq高低音接口
20
21 #define EQ_SECTION_MAX 5 //eq 段数，最大20
22
23 #define TCFG_ABC_DCCS_EQ_ENABLE 1 // ABC DCCS
24 #define TCFG_ABC_UL_EQ_ENABLE 1 // ABC UL
25
26
27 #if (TCFG_EQ_ENABLE == 0)
28 #undef TCFG_EQ_ONLINE_ENABLE
29 #define TCFG_EQ_ONLINE_ENABLE 0
30 #undef TCFG_BT_MUSIC_EQ_ENABLE
31 #define TCFG_BT_MUSIC_EQ_ENABLE 0
32 #undef TCFG_PHONE_EQ_ENABLE
33 #define TCFG_PHONE_EQ_ENABLE 0
34 #undef TCFG_ABC_DCCS_EQ_ENABLE
35 #define TCFG_ABC_DCCS_EQ_ENABLE 0
36 #undef TCFG_ABC_UL_EQ_ENABLE
37 #define TCFG_ABC_UL_EQ_ENABLE 0
38 #endif
39
40 #define TCFG_DRC_ENABLE 0 //DRC
41 #define TCFG_BT_MUSIC_DRC_ENABLE 0 //支持蓝牙音乐DRC
42
43 // ONLINE CONFIG
44 #define TCFG_ONLINE_ENABLE (TCFG_EQ_ONLINE_ENABLE) //是否支持EQ在线调试功能
45 #define TCFG_ONLINE_TX_PORT IO_PORT_DP //EQ调试TX口选择
46 #define TCFG_ONLINE_RX_PORT IO_PORT_DM //EQ调试RX口选择
47 //*****EQ配置*****//

```

例如在 earphne 的 standard sdk 添加 eq_hw_cfg.bin



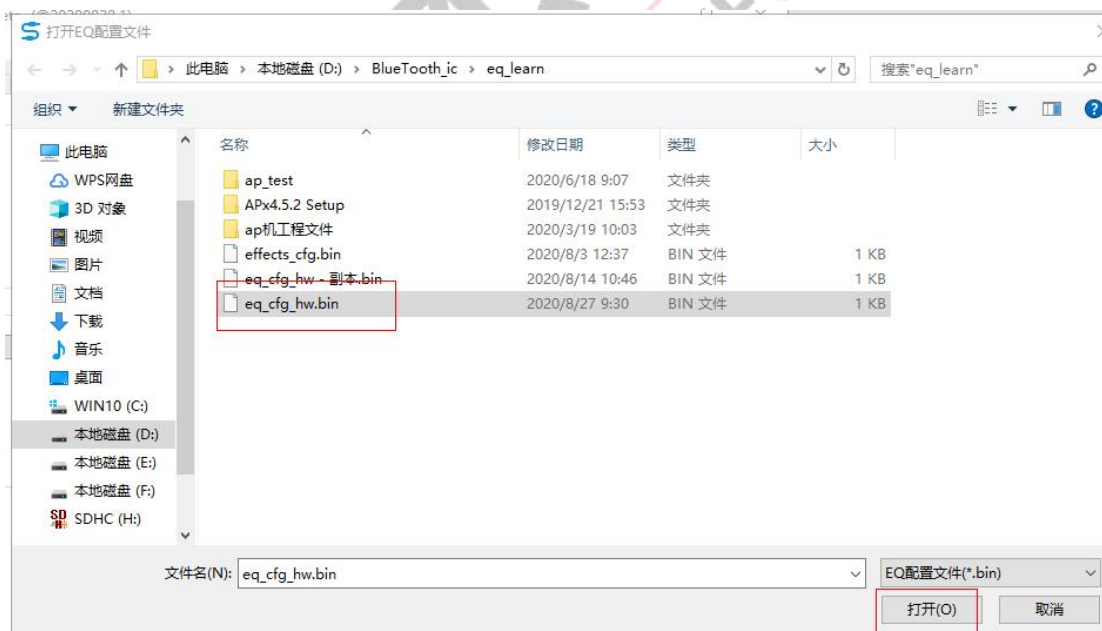
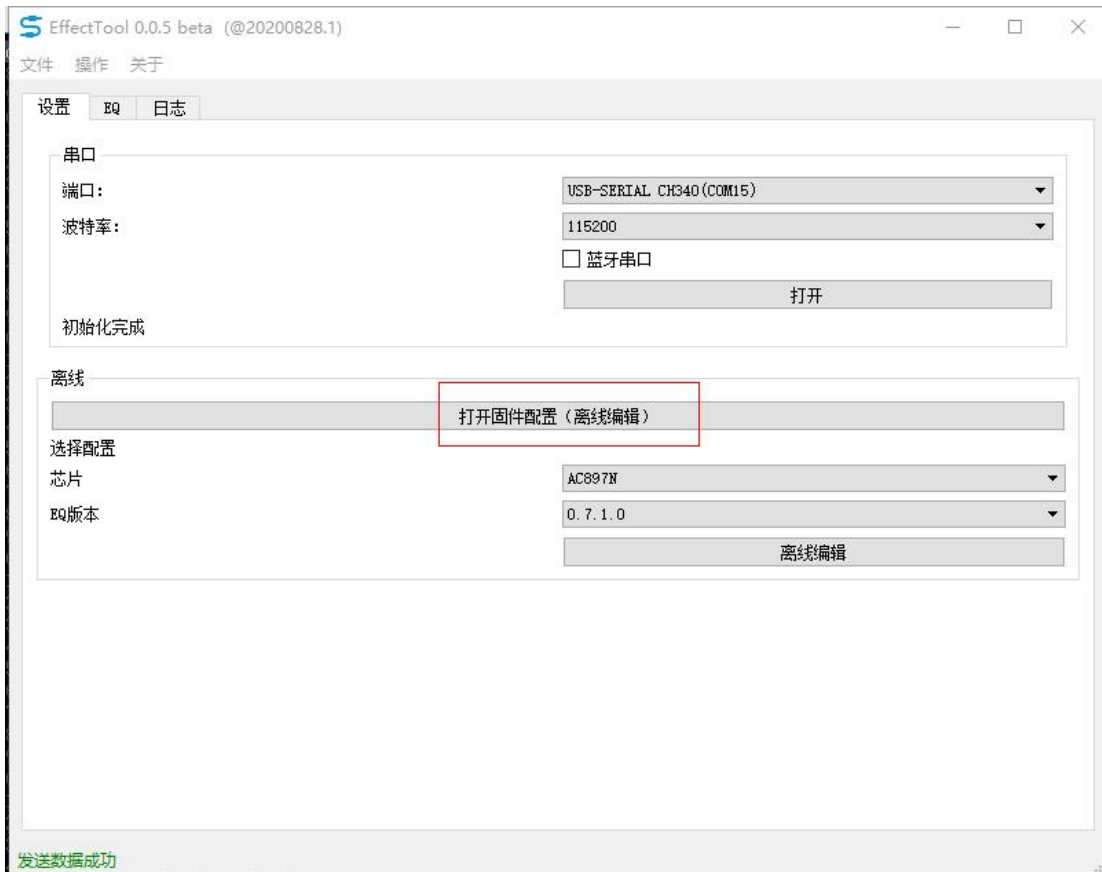
isd download.exe -tonorflash -dev br30 -boot 0x2000 -div8 -wait 300 -uboot uboot.boot -app app.bin cfg_tool.bin
-res tone.cfg pll_code.bin eq_cfg_hw.bin -uboot_compress

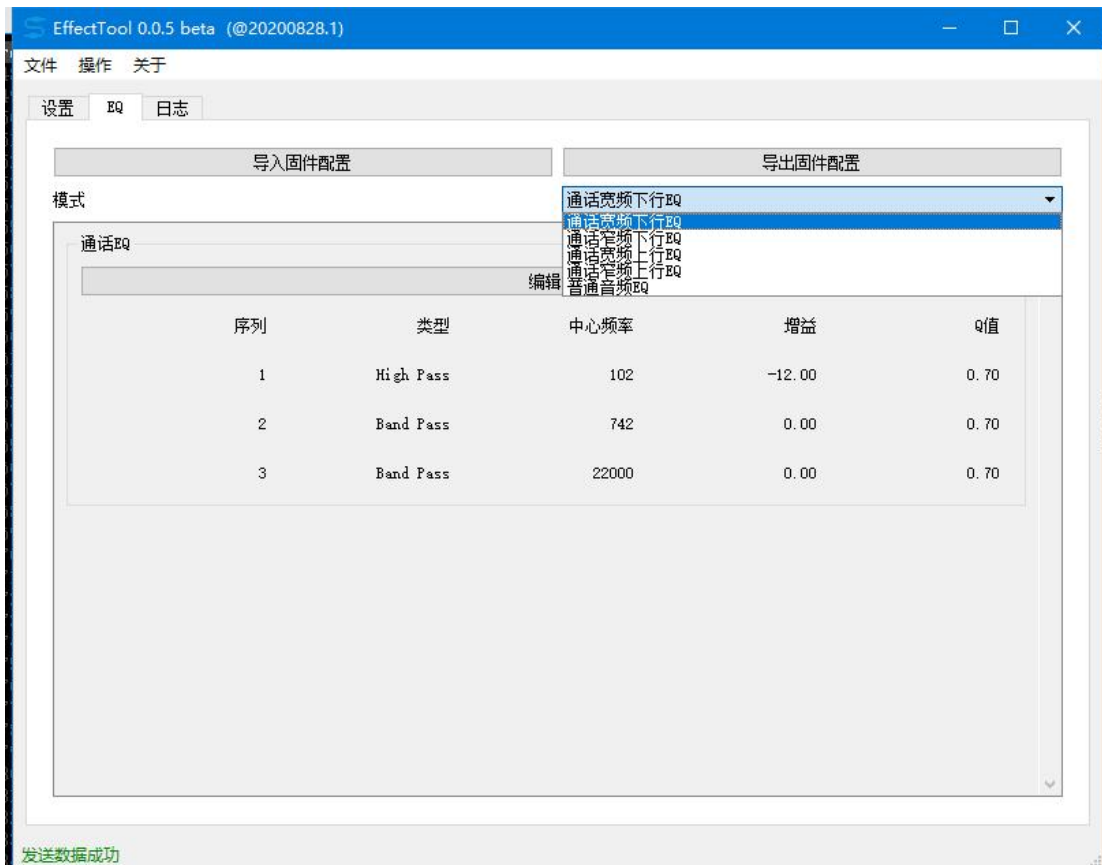
注意这里外部配置的资源一定要放在-res选项中

注意：外部的配置资源一定要放在-res 这个所属项里头，例如 eq_cfg_hw.bin 的资源，因为跟工具的识别项是同步的。

3 eq_hw_cfg.bin 文件重编辑

选中打开固件配置（离线编辑），选中 eq_hw_cfg.bin 文件，就会弹出与在线调试一致的界面，此时可以对已有的 eq_hw_cfg.bin 文件进行修改，修改完成，导出固件



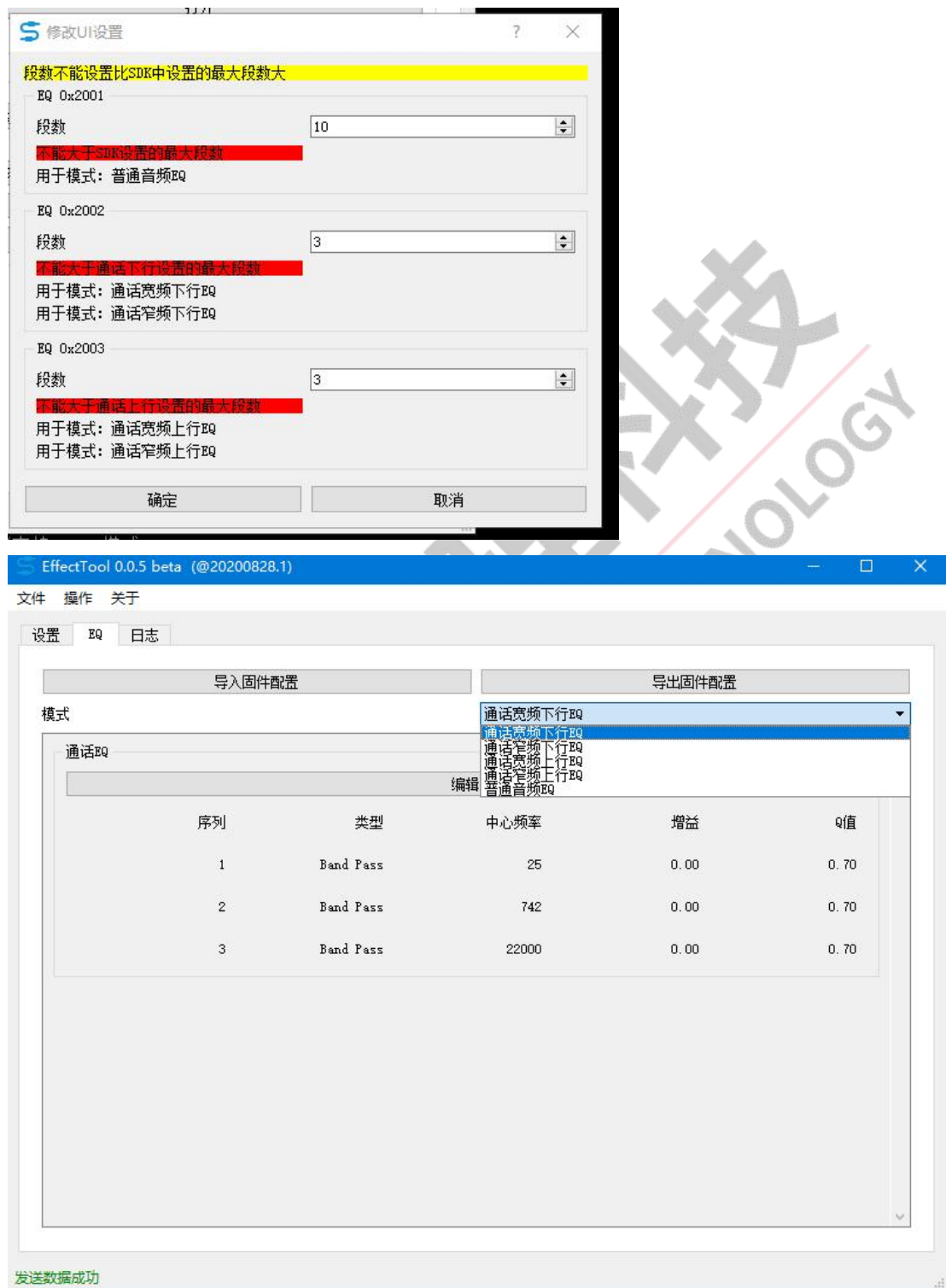


4 打开工具，eq 离线调试编辑

4.1 选中对应芯片版本、以及 eq 版本，点击离线编辑



4.2 弹出设置 eq 段数的界面，设置目标段数，点击确定，会弹出 eq 离线编辑界面。编辑完成导出固件



5 蓝牙串口调试 eq

通讯成功后，使用方式参照普通串口调试。此处说明，如何使用电脑蓝牙与小机进行连接配置。

5.1 软件修改

使用蓝牙串口调试 eq, sdk 需要使能宏定义 APP_ONLINE_DEBUG

```
384
385 #define APP_ONLINE_DEBUG 1 //在线APP调试, 发布默认不开
386 #if (APP_ONLINE_DEBUG && TCFC_ONLINE_ENABLE) //使能蓝牙串口在线调试时, 关闭普通串口在线调试
387 #undef TCFC_ONLINE_ENABLE
388 #define TCFC_ONLINE_ENABLE 0
389 #endif
390
```

5.2 蓝牙串口连接说明

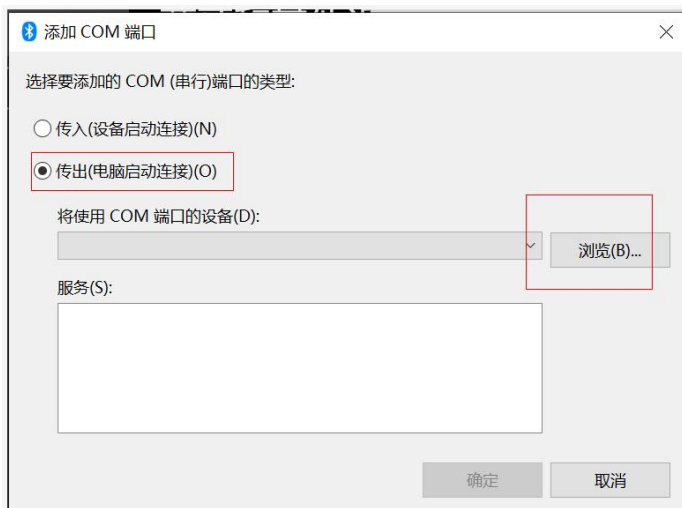
5.2.1 连上蓝牙



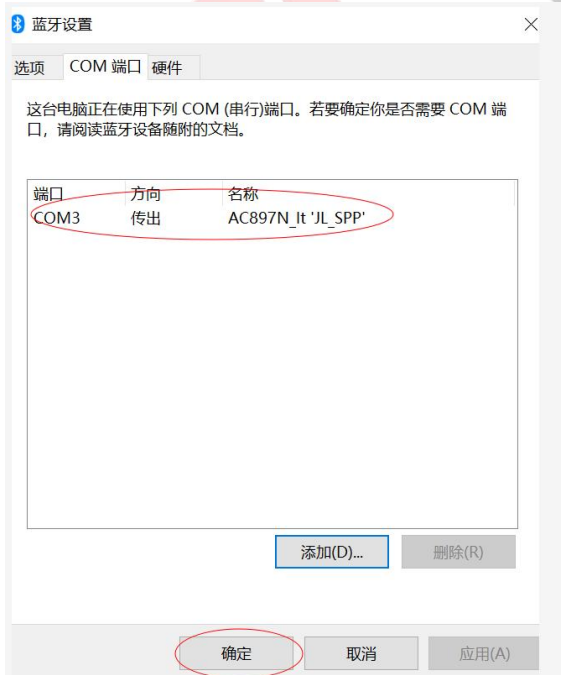
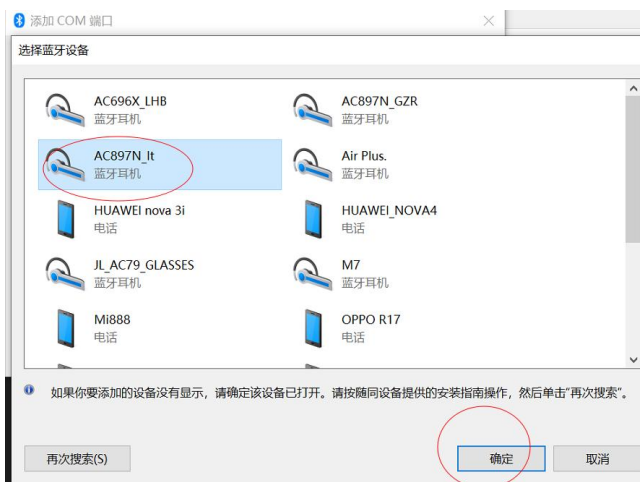
5.2.2 更多蓝牙配置



5.2.3 传出（电脑启动连接）

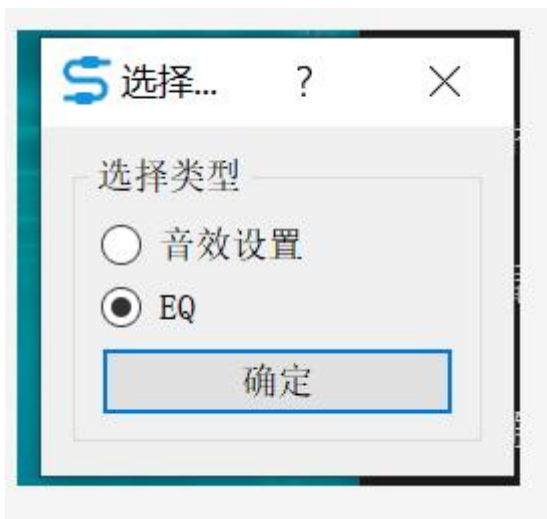


5.2.4 选中目标蓝牙名



5.2.5 打开 eq 工具

选 eq



5.2.6 选中蓝牙串口、选刚才传出的 com 口，波特率默认 115200



7.3 DAC 配置

7.3.1 DAC 输出通道配置

TCFG_AUDIO_DAC_CONNECT_MODE 决定方案需要使用多少个 DAC 声道，TCFG_AUDIO_DAC_MODE 决定 DAC 是差分输出模式还是单端输出模式。

.output:

- 1、DAC_OUTPUT_MONO_L : (1)差分接: DACLP DACLN 差分接 (2) 单端接: DACLP 单端接地
- 2、DAC_OUTPUT_MONO_R : (1)差分接: DACRP DACRN 差分接 (2) 单端接: DACRP 单端接地
- 3、DAC_OUTPUT_LR : DACLP/RP 共地
- 4、DAC_OUTPUT_MONO_LR_DIFF : DACLP DACRP 差分接 (mode 要选单端输出, 选差分 N 端也会打开导致功耗大)

注意: 需要根据具体的硬件接法, 配置 **TCFG_AUDIO_DAC_CONNECT_MODE** 宏
音箱和手表的配置例子如下(在 audio_way_dac.c):

```
const struct audio_dac_platform_data sound_dac_data = {
    .output = TCFG_AUDIO_DAC_CONNECT_MODE,
    .mode = TCFG_AUDIO_DAC_MODE,
    .vcm_cap_en = 1,
    .power = {
        .ldo_isel = 3,
        .ldo_fb_isel = 3,
        .lpf_isel = 0xf, // 0x8,
        .vcmo_en = 0, // 1,
        .vcmo_always_on = 0,
        .vcm_risetime = 0,
        .trim_poweron_time = 500,
        .trim_value = audio_dac_trim_value,
        .trim_begin = NULL,
        .trim_end = audio_dac_trim_end_handler,
    },
    .codec = {
        .dsm_clk = DAC_DSM_6MHz,
    },
};
```

耳机的配置例子如下(在板级配置文件, 如 board_JL7012f_demo.c):

```
struct dac_platform_data dac_data = {
    .mode = TCFG_AUDIO_DAC_MODE,
    .ldo_id = TCFG_AUDIO_DAC_LDO_SEL,
    .ldo_volt = TCFG_AUDIO_DAC_LDO_VOLT,
    .pa_mute_port = TCFG_AUDIO_DAC_PA_PORT,
    .vcmo_en = 0,
    .pa_mute_value = 1,
    .output = TCFG_AUDIO_DAC_CONNECT_MODE,
    .lpf_isel = 0xf,
    .sys_vol_type = SYS_VOL_TYPE,
    .max_ana_vol = MAX_ANA_VOL,
    .max_dig_vol = MAX_DIG_VOL,
    /* .dig_vol_tab = (s16 *)dig_vol_table,
    .vcm_cap_en = 1,
    能力下降,ADC性能下降
    #if (SYS_VOL_TYPE == VOL_TYPE_AD)
        .digital_gain_limit = 16384,
    #endif // #if (SYS_VOL_TYPE == VOL_TYPE_AD)
};
```

7.3.2 DAC 高低压模式选择

JL701N 系列芯片支持五种 DAC 模式，如下：

```
/* DAC模式选择
#define DAC_MODE_L_DIFF (0) // 低压差分模式，适用于低功率差分耳机，输出幅度 0~2Vpp
#define DAC_MODE_H1_DIFF (1) // 高压1档差分模式，适用于高功率差分耳机，输出幅度 0~3Vpp，VDDIO >= 3.0V
#define DAC_MODE_H1_SINGLE (2) // 高压1档单端模式，适用于高功率单端PA音箱，输出幅度 0~1.5Vpp，VDDIO >= 3.0V
#define DAC_MODE_H2_DIFF (3) // 高压2档差分模式，适用于高功率差分PA音箱，输出幅度 0~5Vpp，VDDIO >= 3.3V
#define DAC_MODE_H2_SINGLE (4) // 高压2档单端模式，适用于高功率单端PA音箱，输出幅度 0~2.5Vpp，VDDIO >= 3.3V
```

DACVDD 的档位在驱动中已固定，外部不可配

7.3.3 DAC 输出幅度

DAC 输出幅度和以下配置有关：

(1) DAC 模式选择,可选模式 和 dac 输出幅度范围如下：

// 低压差分模式，适用于低功率差分耳机，输出幅度 0~2Vpp

```
#define DAC_MODE_L_DIFF (0)
```

// 高压 1 档差分模式, 适用于高功率差分耳机，输出幅度 0~3Vpp，VDDIO >= 3.0V

```
#define DAC_MODE_H1_DIFF (1)
```

// 高压 1 档单端模式, 适用于高功率单端 PA 音箱，输出幅度 0~1.5Vpp，VDDIO>=3.0V


```
#define DAC_MODE_H1_SINGLE (2)
```

```
// 高压 2 档差分模式, 适用于高功率差分 PA 音箱, 输出幅度 0~5Vpp , VDDIO>=3.3V
```

```
#define DAC_MODE_H2_DIFF (3)
```

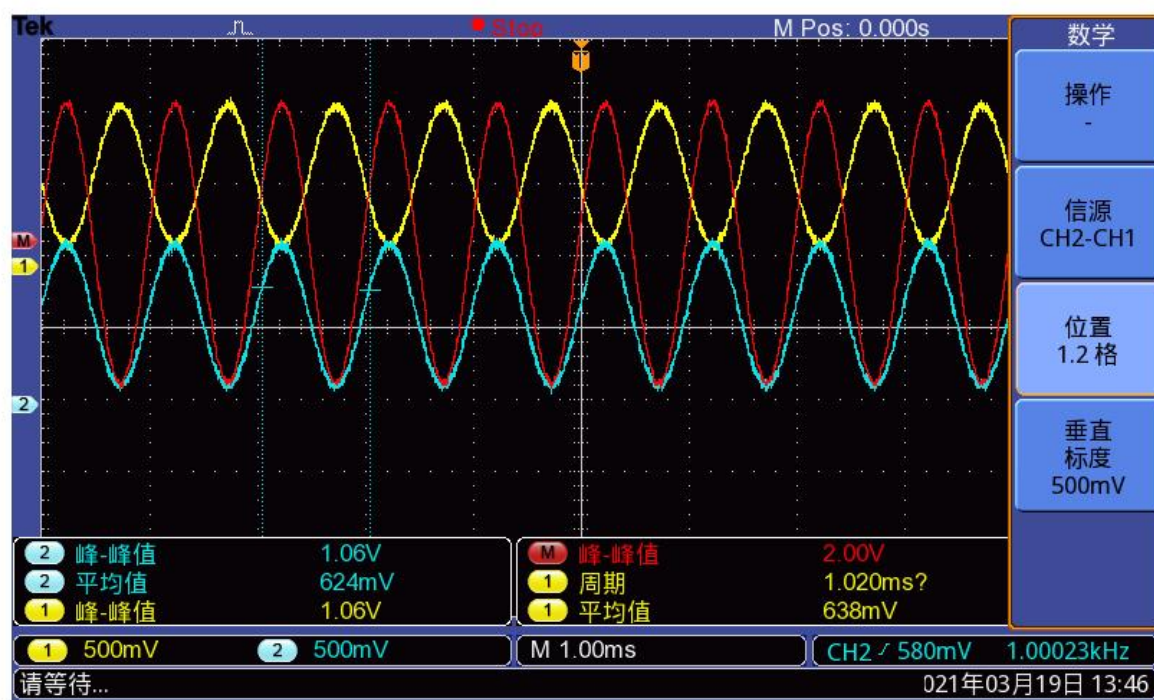
```
// 高压 2 档单端模式, 适用于高功率单端 PA 音箱, 输出幅度 0~2.5Vpp , VDDIO>=3.3V
```

```
#define DAC_MODE_H2_SINGLE (4)
```

(2) DAC 增益等级, 目前 JL701N 音量调节仅支持固定模拟调节数字的方式, 固定的模拟增益档位设置如下:

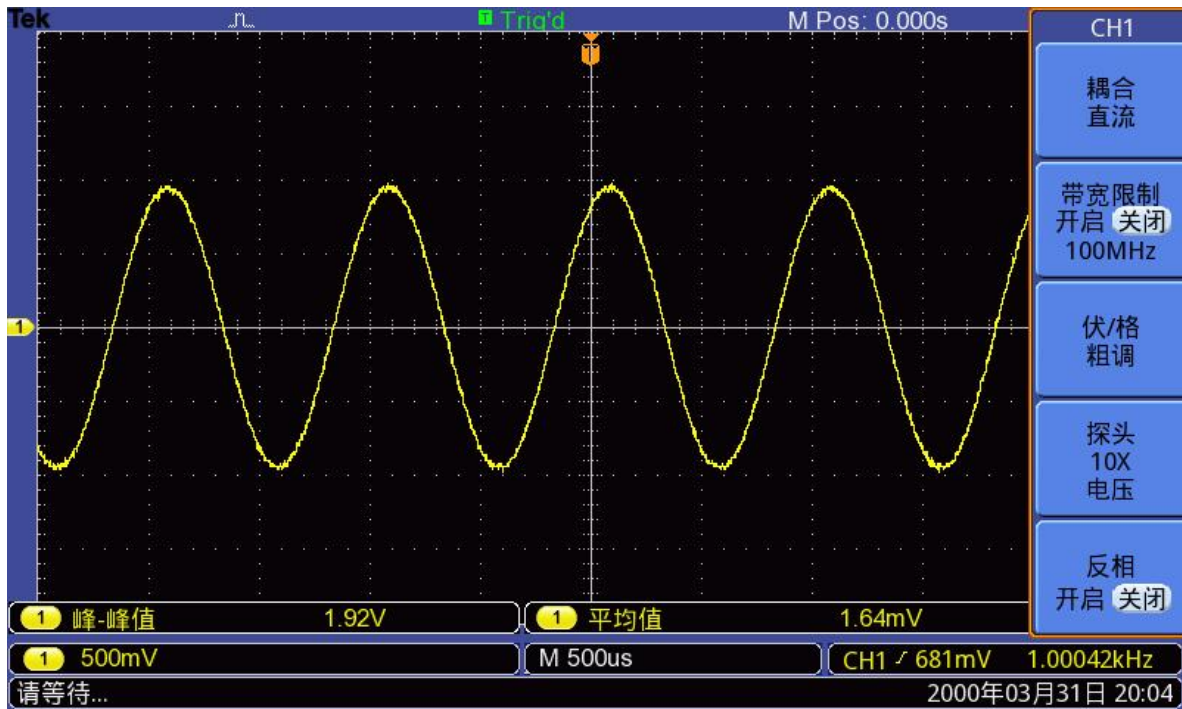
```
#define MAX ANA VOL (3) //系统最大模拟音量 (0-3)
```

关于差分信号幅度的测量, 可以将示波器两个通道正端分别单端接输出的 P 端和 N 端, 负端接地, 然后使用 math 功能显示差分信号, 实际测试如下:

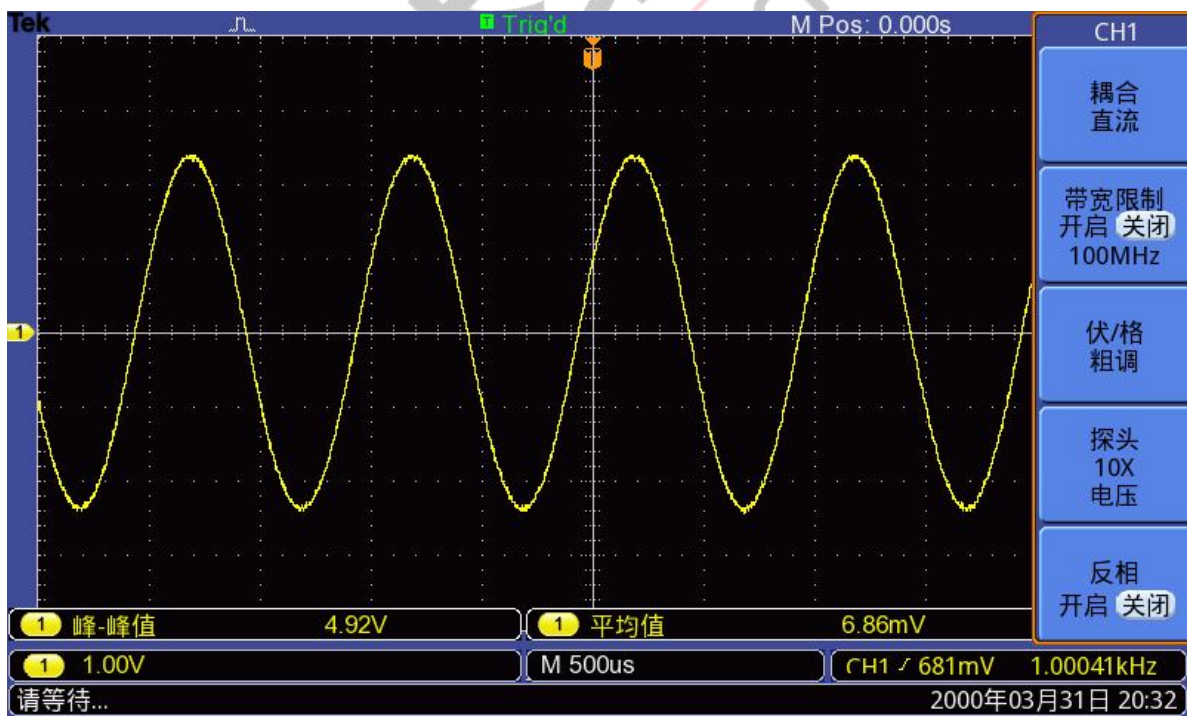


或者板子使用电池供电, 不要与示波器共地, 也可以直接使用示波器一个通道正端接 P, 负端接 N, 可以直接量到差分信号。

DAC 模式选择 DAC_MODE_L_DIFF 时, 实测 DAC 最大输出幅度如下图:



DAC 模式选择 DAC_MODE_H2_DIFF 时，实测 DAC 最大输出幅度如下图:



7.3.4 DAC 使用范例

范例路径: cpu/br28/audio_demo/audio_dac_demo.c

使用方法: 把 void audio_dac_demo_open(void) 函数放在 app_main() 中 audio_dec_init() 后调用即可。demo 为持续输出一个正弦信号。

使用示例:

```
log_info("app_main\n");
app_var.start_time = timer_get_ms();

#if (defined(CONFIG_MEDIA_NEW_ENABLE) || (defined(CONFIG_MEDIA_DEVELOP_ENABLE)))
/*解码器*/
audio_enc_init();
audio_dec_init();
#endif

extern void audio_adc_mic_demo_open(u8 mic_idx, u8 gain, u16 sr, u8 mic_2_dac);

extern void audio_dac_demo_open(void);
/* audio_adc_mic_demo_open(BIT(0)|BIT(1)|BIT(2)|BIT(3),10,16000,1); */

audio_dac_demo_open();

while(1)
{
    os_time_dly(100);
    audio_adda_dump(); //打印所有的dac,adc寄存器
    audio_gain_dump();
}
```

7.4 ADC 配置

7.4.1 ADC 使用范例

范例路径:

耳机 sdk:

cpu/br28/audio_demo/audio_adc_demo.c

音箱和手表 sdk:

cpu/br28/audio_enc/audio_adc_demo.c

使用示例:

```
struct intent it;

log_info("app_main\n");
app_var.start_time = timer_get_ms();

#if (defined(CONFIG_MEDIA_NEW_ENABLE) || (defined(CONFIG_MEDIA_DEVELOP_ENABLE)))
/*解码器*/
audio_enc_init();
audio_dec_init();
#endif

extern void audio_adc_mic_demo_open(u8 mic_idx, u8 gain, u16 sr, u8 mic_2_dac);

/* extern void audio_dac_demo_open(void); */
audio_adc_mic_demo_open(BIT(0) | BIT(1) | BIT(2) | BIT(3), 10, 16000, 1);

/* audio_dac_demo_open(); */

while(1)
{
    os_time_dly(100);
    audio_adda_dump(); //打印所有的dac,adc寄存器
    audio_gain_dump();
}

#ifdef BT_DUT_INTERFERE
void audio_demo(void);
audio_demo();
#endif/*BT_DUT_INTERFERE*/
#ifdef BT_DUT_ADC_INTERFERE
void audio_adc_mic_dut_open(void);
audio_adc_mic_dut_open();
#endif/*BT_DUT_ADC_INTERFERE*/
```

7.4.2 通话过程 mic 数据获取

耳机文件路径: cpu/br28/audio/audio_enc.c

音箱和手表文件路径: cpu/br28/audio_enc/audio_enc.c

函数接口:

- (1) ADC_MIC: adc_mic_output_handler
- (2) PDM_MIC: audio_plnk_mic_output
- (3) IIS_MIC: audio_iis_mic_output

7.4.3 省电容 mic 使用

请查阅“省电容 mic 使用手册.docx”，了解省电容 mic 的详细用法和调试方式

7.5 音量配置

7.5.1 音量调节方式

```

1.  /*
2.  *系统音量类型选择
3.  *软件数字音量是指纯软件对声音进行运算后得到的
4.  *硬件数字音量是指 dac 内部数字模块对声音进行运算后输出
5.  *选择软件数字音量或者软件数字音量会固定模拟音量,
6.  *固定的模拟音量值由 MAX_ANA_VOL 决定
7.  */
8.  #define VOL_TYPE_DIGITAL      0    //软件数字音量
9.  #define VOL_TYPE_AD           2    //联合音量(模拟数字混合调节 暂不支持)
10. #define VOL_TYPE_DIGITAL_HW   3    //硬件数字音量
11. #define VOL_TYPE_DIGGROUP     4    //独立通道软件数字音量
12. #define SYS_VOL_TYPE          VOL_TYPE_DIGITAL_HW
13. //每个解码通道都开启数字音量管理,音量类型为 VOL_TYPE_DIGGROUP 时要使能
14. #define SYS_DIGVOL_GROUP_EN    DISABLE

```

目前 SDK 支持调节软件数字音量、硬件数字音量以及数字音量组（每个解码通道使用独立的数字音量管理）。

模拟音量等级为固定值，固定值由 MAX_ANA_VOL 决定，这个值也决定了 DAC 最大输出信号幅度，DIG_VOL_MAX_VALUE 决定数字量的最大输出幅值，DIG_VOL_STEP 决定每级音量的递减 db 值，如下如所示(在 audio_config.h 中配置)

```

1.  #define MAX_ANA_VOL            (3) //系统最大模拟音量 (0-3)
2.
3.  #if (TCFG_AUDIO_DAC_MODE == DAC_MODE_L_DIFF)
4.  #define DIG_VOL_MAX_VALUE      (-5.0f) // 数字音量最大值(单位:db)
5.  #elif (TCFG_AUDIO_DAC_MODE == DAC_MODE_H1_DIFF)
6.  #define DIG_VOL_MAX_VALUE      (-8.0f) // 数字音量最大值(单位:db)
7.  #else
8.  #define DIG_VOL_MAX_VALUE      (0.0f) // 数字音量最大值(单位:db)

```

```
9. #endif
```

```
10. #define DIG_VOL_STEP          (3.0f)  // 逐级递减差值(单位:dB)
```

7.5.2 K 歌音箱音量配置指引

k 歌音箱一般需要将音乐音量与麦克风通路音量分开调节，通常会使用数字音量组进行音量管理。

● 系统音量配置

参考 7.5.1，配置音量类型为数字音量组。

```
1. #define SYS_VOL_TYPE          VOL_TYPE_DIGGROUP  // VOL_TYPE_DIGITAL_HW
2. //每个解码通道都开启数字音量管理,音量类型为 VOL_TYPE_DIGGROUP 时要使能
3. #define SYS_DIGVOL_GROUP_EN    ENABLE
```

● 麦克风通路音量配置

在 effect_reg.c 中，增加 MIC_EFFECT_CONFIG_DVOL，使能麦克风通路独立数字音量组。

```
1. #define MIC_EFFECT_CONFIG      0 \
2.                                | BIT(MIC_EFFECT_CONFIG_NOISEGATE) \
3.                                | BIT(MIC_EFFECT_CONFIG_EQ) \
4.                                | BIT(MIC_EFFECT_CONFIG_HOWLING) \
5.                                | BIT(MIC_EFFECT_CONFIG_HOWLING_TRAP) \
6.                                | BIT(MIC_EFFECT_CONFIG_DVOL) \
```

7.6 编解码配置

7.6.1 解码声道配置

解码声道配置根据 DAC 输出模式做自动适配，也可根据需要设置与 DAC 输出相匹配的声道。

JL701N 支持声道如下:

```
enum audio_channel {  
    AUDIO_CH_LR = 0,           //立体声  
    AUDIO_CH_L,                //左声道 (单声道)  
    AUDIO_CH_R,                //右声道 (单声道)  
    AUDIO_CH_DIFF,             //差分 (单声道)  
    AUDIO_CH_DUAL_L,           //双声道都为左  
    AUDIO_CH_DUAL_R,           //双声道都为左右  
    AUDIO_CH_DUAL_LR,          //双声道为左右混合  
  
    AUDIO_CH_MAX = 0xff,  
};
```

设置声道接口统一为:

```
int audio_decoder_set_output_channel(struct audio_decoder *dec, enum audio_channel);
```

耳机使用示例 (在 audio_dec.c 中)

```
        dec->ch = 1;
    }
}
#else
    dac_connect_mode = audio_dac_get_channel(&dac_hdl);
    if (dac_connect_mode == DAC_OUTPUT_LR) {
        channel = AUDIO_CH_LR;
        dec->ch = 2;
    } else {
        channel = AUDIO_CH_DIFF;
        dec->ch = 1;
    }
}
#endif

#if TCFG_APP_FM_EMITTER_EN
    channel = AUDIO_CH_LR;
#endif
if (channel != dec->channel) {
    printf("set_channel: %d\n", channel);
    audio_decoder_set_output_channel(&dec->decoder, channel);
    dec->channel = channel;
}
#if TCFG_EQ_ENABLE&&TCFG_BT_MUSIC_EQ_ENABLE
    if (dec->eq_drc && dec->eq_drc->eq) {
        audio_eq_set_channel(dec->eq_drc->eq, dec->ch);
    }
}
#endif//TCFG_BT_MUSIC_EQ_ENABLE

/* #if TCFG_USER_TWS_ENABLE */
#if AUDIO_SURROUND_CONFIG
    if (dec->sur) {
        audio_surround_set_ch(dec->sur, channel);
    }
}
#endif//AUDIO_SURROUND_CONFIG
/* #endif//TCFG_USER_TWS_ENABLE */

}
```

音箱和手表的使用实例在库中，可参考耳机 SDK 的用法

7.6.2 编解码格式配置

```
//*****
//                                     编解码格式配置(CodecFormats Config)
//*****
//解码格式使能配置(Decode Format Config)
#define TCFG_DEC_MP3_ENABLE           ENABLE
#define TCFG_DEC_WTG2V2_ENABLE        ENABLE
#define TCFG_DEC_G729_ENABLE          ENABLE
#define TCFG_DEC_WMA_ENABLE           ENABLE
#define TCFG_DEC_WAV_ENABLE           ENABLE
#define TCFG_DEC_FLAC_ENABLE          ENABLE
#define TCFG_DEC_APE_ENABLE           ENABLE
#define TCFG_DEC_M4A_ENABLE           ENABLE
#define TCFG_DEC_ALAC_ENABLE          ENABLE
#define TCFG_DEC_AMR_ENABLE           ENABLE
#define TCFG_DEC_DTS_ENABLE           ENABLE
#define TCFG_DEC_G726_ENABLE          DISABLE
#define TCFG_DEC_MIDI_ENABLE          DISABLE
#define TCFG_DEC_MTY_ENABLE           DISABLE
#define TCFG_DEC_SBC_ENABLE           ENABLE
#define TCFG_DEC_PCM_ENABLE           ENABLE
#define TCFG_DEC_CVSD_ENABLE          DISABLE
#define TCFG_DEC_LC3_ENABLE           DISABLE

//编码格式使能配置(Encode Format Config)
#define TCFG_ENC_CVSD_ENABLE          ENABLE
#define TCFG_ENC_MSBC_ENABLE          ENABLE
#define TCFG_ENC_MP3_ENABLE           ENABLE
#define TCFG_ENC_ADPCM_ENABLE         DISABLE
#define TCFG_ENC_PCM_ENABLE           DISABLE
#define TCFG_ENC_SBC_ENABLE           DISABLE
#define TCFG_ENC_OPUS_ENABLE          DISABLE
#define TCFG_ENC_SPEEX_ENABLE         DISABLE
#define TCFG_ENC_LC3_ENABLE           DISABLE
```

1、编解码使能配置

这里需要根据具体方案需求进行配置，既能满足方案需求，又不额外使能不需要的解码格式，减少方案代码量。

2、提示音配置

提示音播放支持的格式跟普通解码一致。

【注意】提示音是否支持叠加播放是单独配置的。如果提示音播放参数 `preemption = 0`，但是对应格式的提示音叠加播放配置 `DISABLE`，则默认以打断方式播放。比如默认是支持 `wts` 格式解码，以及 `wts` 格式提示音叠加，则可以在播放 `wts` 格式提示音的时候，根据需要配置 `preemption` 参数接口。

7.7 音频同步配置

1) 蓝牙音频同步开关宏定义：

audio_dec.c 中,

```
#define AUDIO_CODEC_SUPPORT_SYNC 1
```

2) 蓝牙 A2DP 音频延时配置:

lib_btctler_config.c 中,

普通延时模式的配置项:

```
const int CONFIG_A2DP_DELAY_TIME = 340 ;//200;
```

低延时模式的配置项:

```
const int CONFIG_A2DP_DELAY_TIME_LO = 100;
```

3) 蓝牙 ESCO 音频延时配置:

audio_dec.c 中, esco_decoder_syncts_setup()函数,

通话模式的 ESCO 延时:

```
#define ESCO_DELAY_TIME 60
```

SIRI/手机语音助手的 ESCO 延时:

```
#define ESCO_RECOGNITION_TIME 220
```

4) TWS 提示音音频同步延迟配置:

```
#define TWS_TONE_CONFIRM_TIME 250 /*TWS 提示音音频同步确认时间(也是音频解码主从  
确认超时时间)*/
```

5) 蓝牙音频同步参考时钟选择配置:

```
void bt_audio_sync_nettime_select(u8 base);//0 - 主机, 1 - tws, 2 - BLE
```

7.8 通话算法开发配置

CVP (Clear Voice Process), 即清晰语音处理。主要用在蓝牙通话中, 对上行数据 (即耳机 mic 发给远端手机的数据) 进行回音消除和降噪等处理, 使得语音清晰, 提升通话效果的算法处理。

7.8.1 算法离线开发

有些应用不用通话或者不用蓝牙, 比如只是单纯做一个降噪处理, 可以通过离线开发的方式进

行开发。

实现代码：cpu/br36/audio/audio_cvp_demo.c

7.8.2 第三方算法开发

SDK 发布默认使用原厂开发的 CVP 相关算法进行处理。如有相应算法开发能力的客户，可自行实现对语音的处理。数据流程已经写好，只需实现算法函数函数（audio_aec_run()）即可。具体可以查看实现代码里面的代码注释。

● SDK 配置：

//第三方清晰语音开发使能：使能之后，即可在 audio_aec_demo.c 开发自研算法

```
#define TCFG_CVP_DEVELOP_ENABLE          ENABLE_THIS_MOUDLE
```

● 实现代码：earphone/aec/br36/audio_aec_demo.c

● 实现工程：JL700N_CVP.cbp

7.8.3 通话参数调试

打开杰理 SDK 工具/编译前配置工具/蓝牙配置：通话参数配置



也可以选择在线调试参数，sdk 配置如下：

```

// *****
//          aec 工具在线调试
// *****
#define TCFG_AEC_TOOL_ONLINE_ENABLE          //aec 在线调试使能，使用蓝牙串口调试，需要打开宏 APP_ONLINE_DEBUG

```

配合安卓系统应用程序（Audio Tools），实现通过过程在线调试。调试完毕，再将配置参数更新到上面 SDK 工具里面的通话参数配置即可。

7.8.4 双 mic 降噪 (ENC)

(1) 双 mic 降噪配置

双 mic 降噪功能相关配置有以下宏定义:

a. 使能双 mic 降噪功能: sdk 板级配置头文件 (比如: board_ad697n_demo_cfg.h)

```
/*ENC(双mic降噪)使能*/
#define TCFG_AUDIO_DUAL_MIC_ENABLE ENABLE_THIS_MOUDLE
```

b. 数据导出调试使能: **调试使用, 正常使用需要关闭**

```
/*Audio数据导出配置:通过蓝牙spp导出或者sd写卡导出*/
#define AUDIO_DATA_EXPORT_USE_SD 1
#define AUDIO_DATA_EXPORT_USE_SPP 2
#define TCFG_AUDIO_DATA_EXPORT_ENABLE AUDIO_DATA_EXPORT_USE_SPP
```

c. 通话在线调试功能: 单 mic/双 mic 均支持

```
/*通话参数在线调试*/
#define TCFG_AEC_TOOL_ONLINE_ENABLE DISABLE_THIS_MOUDLE
```

d. 指标测试功能: 如果需要随时可以测试指标, 则改宏定义需要常开

```
/*双mic降噪DUT测试模式, 配合设备测试mic频响和降噪量*/
#define TCFG_AUDIO_DMS_DUT_ENABLE ENABLE_THIS_MOUDLE
```

e. 数据处理流程

apps/earphone/aec/br28/audio_aec_dms.c

实现流程:

(1) 测试仪器通过蓝牙和耳机连接, 然后通过 spp 发对应的命令给小机处理

(2) 默认的命令有:

DMS_OUTPUT_SEL_DEFAULT	输出算法处理结果, 用来测试 ENC 效果
DMS_OUTPUT_SEL_MASTER	输出主 mic 的原始数据: 测试 mic 的频响
DMS_OUTPUT_SEL_SLAVE	输出副 mic 的原始数据: 测试 mic 的频响

(3) 通过接口 audio_aec_output_sel() 控制 CVP 模块的输出, 具体参数传递, 请看代码注释

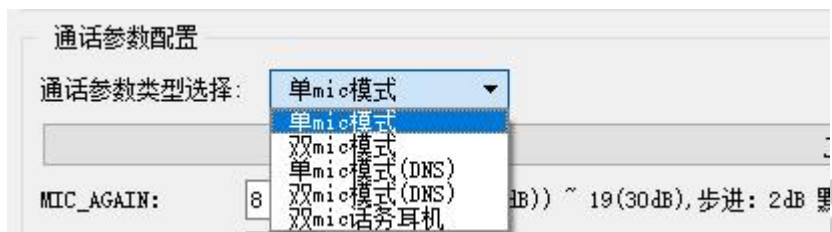
7.8.5 神经网络降噪 DNS

(1) 神经网络降噪使能

在板级头文件, 根据需要, 选择降噪模式, ANS 为传统降噪, DNS 为神经网络降噪:


```
/*通话降噪模式配置*/
#define CVP_ANS_MODE 0 /*传统降噪*/
#define CVP_DNS_MODE 1 /*神经网络降噪*/
#define TCFG_AUDIO_CVP_NS_MODE CVP_DNS_MODE
```

(2) 神经网络降噪参数配置



根据单 MIC 还是双 MIC，ANS 还是 DNS 进行模式选择，配置完参数，点击保存（**仅保存被选择的模式的配置**）。

7.8.6 双 MIC 话务耳机

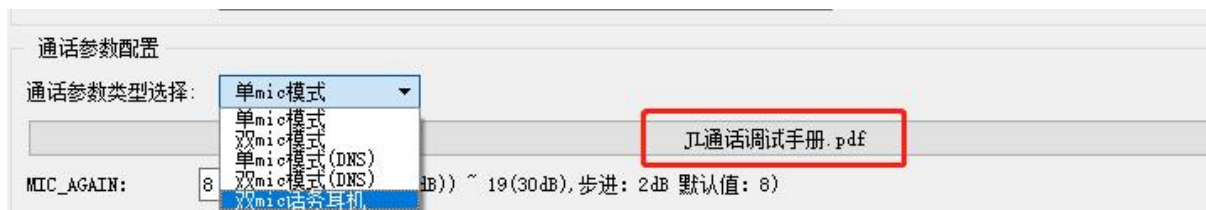
(1) 话务耳机功能使能

```
/*
 * ENC(双mic降噪) 配置
 * 双mic降噪包括DMS_NORMAL和DMS_FLEXIBLE，在使能TCFG_AUDIO_DUAL_MIC_ENABLE
 * 的前提下，根据具体需求，选择对应的DMS模式
 */
/*ENC(双mic降噪) 使能*/
#define TCFG_AUDIO_DUAL_MIC_ENABLE ENABLE_THIS_MOUDLE

/*DMS模式选择*/
#define DMS_NORMAL 1 //普通双mic降噪(mic距离固定)
#define DMS_FLEXIBLE 2 //适配mic距离不固定且距离比较远的情况，比如头戴式话务耳机
#define TCFG_AUDIO_DMS_SEL DMS_FLEXIBLE
```

先使能双 mic 降噪功能，再选择 DMS 类型：DMS_FLEXIBLE

(2) 话务耳机参数配置



选择“双 mic 话务耳机”模式，具体调试指引，请查阅工具内置的调试手册。

7.9 语音识别配置

1、离线语音介绍说明

目前离线语音识别支持**远程**语音识别和**近场**语音识别，两个场景的语音识别的区别是有效识别距离不一样，远程语音识别的有效识别距离是 5 米，近场的语音识别的有效识别距离是 0.25 米。

注意：**远程**语音识别识别需要**获取授权才能正常使用**，没有授权每次可以体验 10 分钟。

2.1、**近场**离线语音支持的关键词说明

关键词	模式
小杰小杰/小杰同学	唤醒手机语音助手/Siri
播放音乐	多媒体（通话不可用）
停止播放	
暂停播放	
增大音量	
减小音量	
上一首	
下一首	
打开降噪	ANC 模式（通话不可用）
关闭降噪	
打开通透	
接听电话	通话来电模式
挂断电话	

2.2、**远场**离线语音支持的关键词说明

关键词	模式
播放音乐	多媒体（通话不可用）
暂停播放	
增大音量	
减小音量	
上一首	
下一首	

3、离线语音使能配置

在对应的板级配置文件中，离线语音识别的配置如下：

```

1.  //*****//
2.  //                      Audio Smart Voice                      //
3.  //*****//
4.  #define TCFG_SMART_VOICE_ENABLE          ENABLE_THIS_MOUDLE
5.  #define TCFG_KWS_VOICE_EVENT_HANDLE_ENABLE  TCFG_SMART_VOICE_ENABLE //语音事件处理流程开关
6.  #if TCFG_SMART_VOICE_ENABLE
7.  #define TCFG_VAD_LOWPPOWER_CLOCK          VAD_CLOCK_USE_PMU_STD12M
8.  #else
9.  #define TCFG_VAD_LOWPPOWER_CLOCK          VAD_CLOCK_USE_LRC
10. #endif/*TCFG_AUDIO_SMART_VOICE_ENABLE*/
11. #if ((defined TCFG_KWS_VOICE_RECOGNITION_ENABLE) && (TCFG_KWS_VOICE_RECOGNITION_ENABLE))
12. #define TCFG_CALL_KWS_SWITCH_ENABLE        DISABLE_THIS_MOUDLE
13. #else
14. #define TCFG_CALL_KWS_SWITCH_ENABLE        DISABLE_THIS_MOUDLE
15. #endif
16.
17. /*识别语言选择*/
18. #define KWS_CH          1    /*中文*/
19. #define KWS_INDIA_EN    2    /*印度英语*/
20. #define KWS_FAR_CH      3    /*音箱中文*/
21. #define TCFG_AUDIO_KWS_LANGUAGE_SEL        KWS_FAR_CH

```

3.1、使能 TCFG_SMART_VOICE_ENABLE 打开离线语音识别功能。

3.2、配置 TCFG_AUDIO_KWS_LANGUAGE_SEL 选择远程语音识别，选择 KWS_CH 为近场语音识别，选择 KWS_FAR_CH 为远程语音识别。

4、播歌模式离线语音关键词识别配置

在 lib_media_config.c 中，根据需求进行关键词的识别阈值配置，普通模式的为 CONFIG_KWS_MULTIO_CONFIGDENCE[]，通话模式未 CONFIG_KWS_CALL_CONFIDENCE[]

```
/*杰理KWS关键词识别*/
#if TCFG_SMART_VOICE_ENABLE
const int CONFIG_KWS_WAKE_WORD_MODEL_ENABLE = 0;
const int CONFIG_KWS_MULTI_CMD_MODEL_ENABLE = 1;
#if TCFG_CALL_KWS_SWITCH_ENABLE
const int CONFIG_KWS_CALL_CMD_MODEL_ENABLE = 1;
#else
const int CONFIG_KWS_CALL_CMD_MODEL_ENABLE = 0;
#endif
#ifdef CONFIG_CPU_BR28
const int CONFIG_KWS_RAM_USE_ENABLE = 1;
#else
const int CONFIG_KWS_RAM_USE_ENABLE = 2;
#endif
const int CONFIG_KWS_ONLINE_MODEL = 1;
const float CONFIG_KWS_MULTI_CONFIDENCE[] = {
    0.4, 0.4, 0.4, 0.4, //小杰小杰, 小杰同学, 播放音乐, 停止播放
    0.4, 0.4, 0.4, 0.4, //暂停播放, 增大音量, 减小音量, 上一首
    0.4, 0.4, 0.4, 0.4, //下一首, 打开降噪, 关闭降噪, 打开通透
};

const float CONFIG_KWS_CALL_CONFIDENCE[] = {
    0.4, 0.4, //接听电话, 挂断电话
};
#else
const int CONFIG_KWS_WAKE_WORD_MODEL_ENABLE = 0;
const int CONFIG_KWS_MULTI_CMD_MODEL_ENABLE = 0;
const int CONFIG_KWS_CALL_CMD_MODEL_ENABLE = 0;
const int CONFIG_KWS_ONLINE_MODEL = 0;
#endif
```

7.10 音效配置

7.10.1 音效支持列表

序号	音效名	中文名	备注
1	EQ	均衡器	
2	DRC	动态范围压缩	
3	VBass	虚拟低音	
4	Voice Changer	变声	
5	CrossOver	分频器	
6	Dynamic EQ	动态 EQ	
7	Echo	回声	
8	Plate Reverb	混响	
9	Frequency Shift	啸叫抑制——移频	

10	Howling Suppress	啸叫抑制——陷波	
11	Surround Effect	环绕声	
12	Autotune	电音修音	
13	BassTreble	高低音调节	
14	EnergyDetect	能量检测	
15	PitchSpeed	变速变调	
16	Spectrum	频谱	
17	StereoMix	立体声混合	
18	Vocal Remover	人声消除	
19	NoiseGate	噪声门限	
20	Voice Changer Adv	高级变声	
21	Plate Reverb Adv	高级混响	
22	HarmonicExciter	谐波激励	

7.10.2 在线调试配置

打开板级配置中的在线调试功能，根据需要可自行选择 USB/SPP/UART 通讯方式，下载之后打开调音工具，即可读取 bin 文件，进入在线调音。

```

//*****
//
//          新音箱配置工具 && 调音工具
//*****
//*****
#define TCFG_CFG_TOOL_ENABLE          DISABLE          //是否支持音箱在线配置工具
#define TCFG_EFFECT_TOOL_ENABLE      1 //DISABLE      //是否支持在线音效调试,使能该项还需使能EQ总使能TCFG_EQ_ENABL,
#define TCFG_NULL_COMM                0                //不支持通信
#define TCFG_UART_COMM                1                //串口通信
#define TCFG_USB_COMM                 2                //USB通信
#define TCFG_SPP_COMM                 3                //SPP通信
#if (TCFG_CFG_TOOL_ENABLE || TCFG_EFFECT_TOOL_ENABLE)
#define TCFG_COMM_TYPE                TCFG_USB_COMM    //通信方式选择
#else
#define TCFG_COMM_TYPE                TCFG_NULL_COMM
#endif
#define TCFG_ONLINE_TX_PORT            IO_PORT_DP      //UART模式调试TX口选择
#define TCFG_ONLINE_RX_PORT            IO_PORT_DM      //UART模式调试RX口选择
#define TCFG_ONLINE_ENABLE             (TCFG_EFFECT_TOOL_ENABLE) //是否支持音效在线调试功能

```

7.10.3 音效配置

1、JL701N 系列支持 DAC 24bit 输出，需要在板级文件中进行配置：

1. /*
2. *DAC 位宽(bit width)配置


```
3.  *16bit: DAC_16BIT_MODE
4.  *24bit: DAC_24BIT_MODE
5.  * */
6.  #define TCFG_AUDIO_DAC_BIT_WIDTH          DAC_24BIT_MODE
```

注：（1）使能 24bit 输出后，download.c 中会自动配置使用 effect_file_24bit 文件夹中的音效文件：

```
#if TCFG_AUDIO_DAC_BIT_WIDTH
#define EFF_DIR effect_file_24bit
#else
#if BASS_AND_TREBLE_ENABLE
#define EFF_DIR effect_file_tunning
#else
#define EFF_DIR effect_file
#endif
#endif
```

（2）使能 DAC_24BIT_MODE 输出模式后，ADC 默认也设置成 ADC_BIT_WIDTH_24 输入模式。

（3）使能 DAC_24BIT_MODE 输出模式后且录音编码格式设置为 PCM 编码时，录音默认编码 24bit 位宽录音文件。

3、当前提供了 kbox（k 歌音箱）板级，默认使能了以下音效模块：虚拟低音、动态 eq、多带 DRC、Reverb、Echo、变声、谐波激励等。使用 **mic_effect_full_harmonic_exciter.bin** 文件。

```
/*
 * 板级配置选择
 */
// #define CONFIG_BOARD_JL701N_DEMO
#define CONFIG_BOARD_JL701N_KBOX //k歌音箱板级
// #define CONFIG_BOARD_JL701N_SOUND CARD
// #define CONFIG_BOARD_JL701N_A90
// #define CONFIG_BOARD_JL701N_BTEMITTER //该板级暂不支持
// #define CONFIG_BOARD_JL701N_SMARTBOX
// #define CONFIG_BOARD_JL701N_TWS_BOX
// #define CONFIG_BOARD_JL701N_UNISOUND_BOX
// #define CONFIG_BOARD_JL701N_WIRELESS_1TN
// #define CONFIG_BOARD_JL701N_WIRELESS_2T1_DUPLEX
// #define CONFIG_BOARD_JL701N_TEST
```

（1）音乐音效使能


```
1. #define AUDIO_VBASS_CONFIG          ENABLE
2. #define TCFG_DYNAMIC_EQ_ENABLE      ENABLE
3. #define TCFG_MIC_HARMONIC_EXCITER_ENABLE  ENABLE
4. //多带DRC 使能 0:关闭多带DRC, 1: 使能多带DRC 2: 使能多带DRC 并且 多带DRC
   后再做一次全带的drc
5. #define TCFG_AUDIO_MDRC_ENABLE      2
```

(2) 麦克风音效使能

```
1. #define TCFG_MIC_EFFECT_SEL          MIC_EFFECT_REVERB_ECHO
2. #define TCFG_MIC_VOICE_CHANGER_ENABLE  ENABLE
3. #define TCFG_MIC_HARMONIC_EXCITER_ENABLE  ENABLE
4. //使用MIC_EFFECT_REVERB_ECHO 时, 是否使能尾部多带DRC
5. #define TCFG_MIC_EFFECT_MDRC          ENABLE
```

7.10.4 音效模块使用说明

7.10.4.1 人声消除 (Vocal Remover)

使用人声消除模块前, 需要在 sdk 中进行参数的配置, 函数如下:

```
1. / *
2.  * remove_ratio(%): 人声消除比例, 100% 完全消除人声, 0% 不消除人声
3.  * low_freq: 保留的低频截止频率
4.  * high_freq: 保留的高频截止频率
5.  * */
6. void *vocal_remove_open(u8 ch_num, u32 samplerate, u8 remove_ratio, u16 low_freq, u16 high_freq)
```

Chapter 8 客户常见问题反馈解答

Q1:

Q2:

